COMSATS University, Islamabad Pakistan

Aassignment-02

CLO-2

PodCraft – Agentic AI Podcast Episode Planning System
Version 1.0

Submitted By:

Asna Hammad FA23-BDS-051

Awais Ashraf FA23-BDS-053

Supervisor

Dr. Tehseen Riaz Abbasi

Bachelor of Science in Data Science / Software Engineering

(2023-2027)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University, Islamabad Pakistan

PodCraft – Agentic AI Podcast Episode Planning System

A project presented to

COMSATS University, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelors of Science in Data Science / Software Engineering

(2023-2027)

By

Asna Hammad FA23-BDS-051

Awais Ashraf FA23-BDS-053

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Asna Hammad

Awais Ashraf

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (DS) PodCraft was developed by Muhammad **Awais Ashraf (FA23-BDS-053)** and **Asna Hammad (FA23-BDS-051)** under the supervision of **Dr. Tehseen Riaz Abbasi** and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Data Science.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

In the rapidly growing domain of digital content creation, podcasters face significant challenges during pre-production, including time-consuming manual research, unorganized episode structuring, and difficulty in generating engaging questions and hooks. Appointing manual methods for this task involves many problems such as increased preparation time, inconsistency in content quality, lack of structured workflows, no intelligent assistance, and reliance on scattered digital tools. Talking about current planning methods, they use general note-taking applications or basic script editors irrespective of the fact that podcasters need automated research and structured outlines.

To prevent all these problems from prevailing, PodCraft is developed. It is an agentic AI-based podcast planning system, which provides solutions to many of these problems. It is a cross-platform application (web and mobile) which allows podcasters to input their niche or guest information. It monitors different content requirements using intelligent autonomous agents which can research topics based on user input and generate structured episode plans for later reference as well.

PodCraft is a podcast planning system that can be assigned to a content type, as in our case podcast episodes, it works accordingly, analyzes user intent upon receiving niche or guest details, and performs user-defined actions like generating questions, hooks, timestamps, and complete episode outlines for future use.

It is an intelligent system consisting of a React JS web frontend, React Native mobile app, Node JS backend with Express, and MongoDB database. OpenAI API is used for AI agent processing, and Firebase is used for cloud authentication and real-time synchronization. LAN and internet communication are used by users to access the platform and operate the system manually or through automated agents.

Abstract

PodCraft is developed to simplify and automate podcast episode planning using intelligent autonomous agents.
Currently, podcasters rely heavily on manual research, scattered online sources, and unstructured planning methods, which consume significant time and often reduce content quality.
Many creators struggle to organize episode flow, prepare engaging questions, and design strong introductions that retain audience attention. This leads to inconsistent production quality and inefficient preparation workflows.
There is no unified intelligent platform that autonomously plans podcast episodes based on niche and guest information.
PodCraft addresses this gap by using autonomous agents to research topics, generate structured questions, create engaging hooks, and design complete episode outlines. The system aims to automate preparation, improve episode structure, and assist creators in producing professional content efficiently. It also enhances time management by providing timestamped segments and guided content flow.
The project ultimately enables podcasters to focus more on creativity and delivery rather than planning logistics. PodCraft improves productivity and raises the overall quality of podcast production through intelligent automation.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor Dr. Tehseen Riaz Abbasi. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Asna Hammad

Awais Ashraf

Abbreviations

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
FE	Feature
FR	Functional Requirement
JWT	JSON Web Token
LI	Limitation
MVC	Model-View-Controller
SRS	Software Requirements Specification
UI	User Interface
UX	User Experience
WBS	Work Breakdown Structure

Table of Contents

Abstract v

3. Chapter 3: Design and Architecture 1

3.1 System Architecture Overview 1
Conceptual Architecture Diagram 2
Technologies and Services 2
3.2 Architecture Style / Pattern 3
Architecture Pattern Diagram: 4
MVC Justification 5
Component Responsibilities 5
3.3 Design Models 6
3.3.1 Sequence Diagrams: 6
3.3.2 Class Diagram: 25
3.3.3 Activity Diagrams: 27
3.3.4 State Diagrams: 50
3.3.5 UseCase Diagram: 54
3.4 Data Design 55

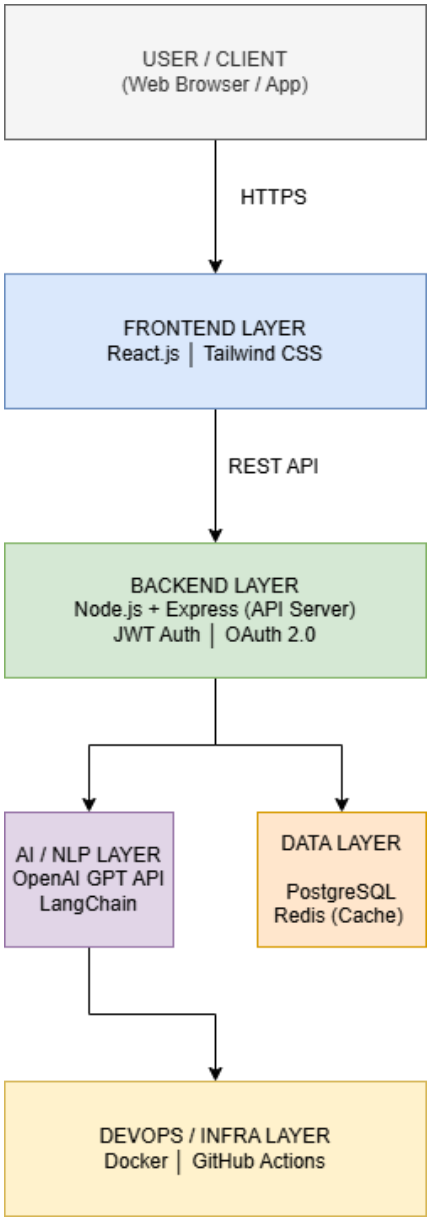
3. Chapter 3: Design and Architecture

This chapter presents the high-level architectural design of the proposed system. It integrates the conceptual system architecture, architectural design patterns, and design models to provide a comprehensive understanding of how the system is structured and implemented. It also includes data design and a data dictionary for full system clarity.

3.1 System Architecture Overview

PodCraft follows a four-layer architecture consisting of Presentation, Application, Agentic AI, and Data layers. The Presentation Layer contains React JS web and React Native mobile applications. The Application Layer uses Node.js with Express to handle routing, authentication, and request validation. The Agentic AI Layer houses five autonomous agents that execute sequentially: Intent Analyzer, Topic Research, Question Generator, Hook Suggester, and Outline Planner. The Data Layer uses MongoDB for primary storage, Firebase for authentication/real-time sync, and Redis for caching research results with a 24-hour TTL (BR-5). All communication between layers occurs over HTTPS using RESTful APIs.

Conceptual Architecture Diagram



Technologies and Services

Table 1 Technologies, Components, and Security Mechanisms Used in the System Architecture

Component	Description	Technology Used	Security Mechanism
[Core] Frontend App	User-facing interface	React.js + Tailwind CSS	SSL/TLS, Input Validation
Backend API	Business logic and routing	Node.js + Express	JWT, OAuth 2.0

[AI] AI Agent Layer	Content generation	OpenAI GPT API	API Key Encryption
AI Orchestration	Pipeline and memory management	LangChain	Secure API Gateway
Database (Primary)	User and episode storage	PostgreSQL	RBAC, Encryption at Rest
Database (Cache)	Session and response caching	Redis	TLS, Access Control
Authentication Service	Login and authorization	JWT + OAuth 2.0	bcrypt, MFA
Admin Panel	User and content moderation	React	2FA, Audit Logging
Payment Gateway	Subscription billing	Stripe	Tokenization, PCI Compliance
Containerization	Consistent deployment	Docker	Image Scanning
CI/CD Pipeline	Automated testing and deployment	GitHub Actions	Secrets Encryption

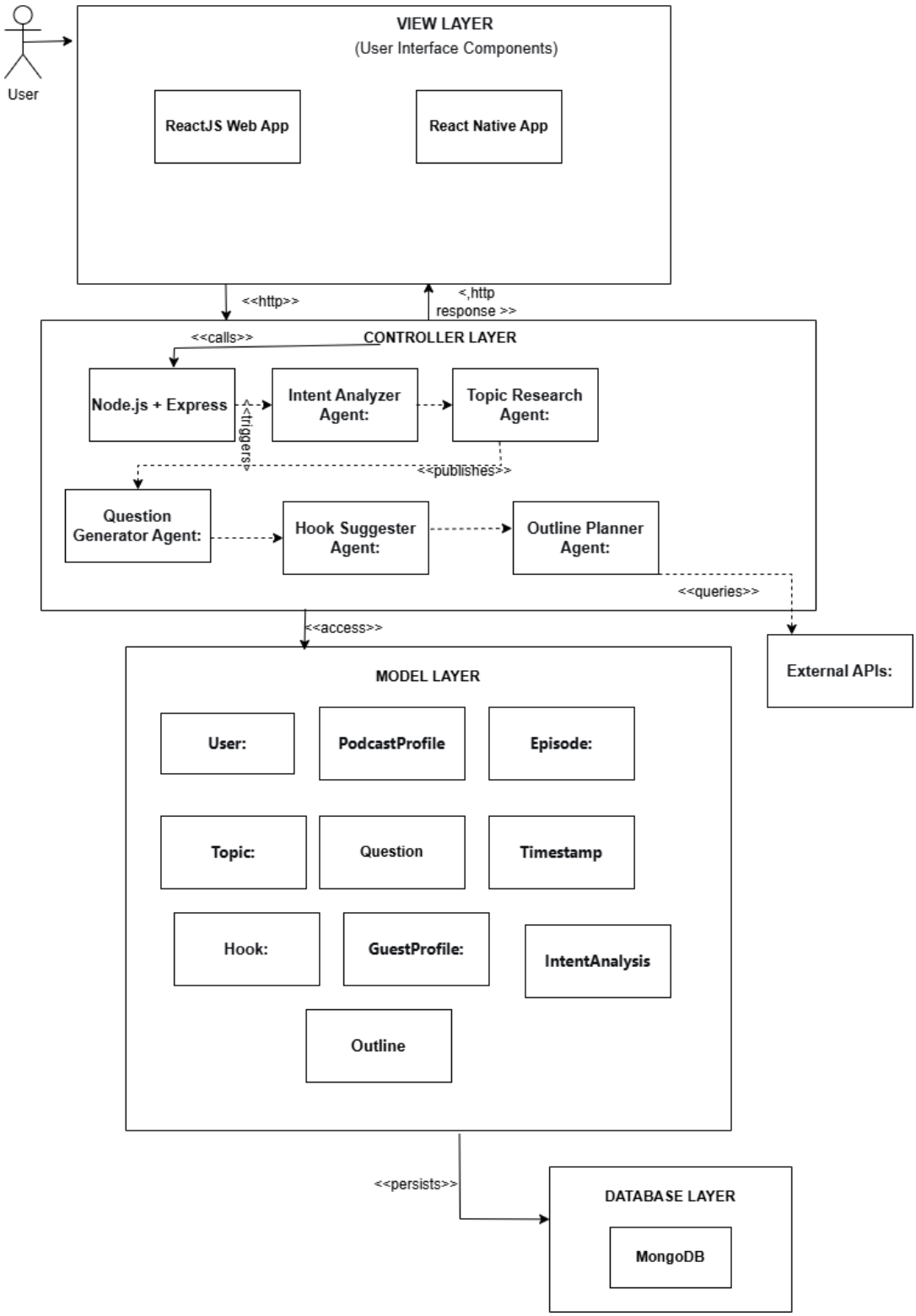
The conceptual architecture of PodCraft is designed around a layered, service-oriented model that separates concerns across the frontend, backend, AI processing, and data persistence layers. At the topmost layer, the React.js frontend provides an intuitive interface through which users can configure podcast projects, input topics, and review AI-generated episode content. All user interactions are transmitted to the backend via RESTful API endpoints exposed by an Express.js server running on Node.js.

The backend orchestrates communication between the user-facing layer and the AI engine. LangChain serves as the orchestration framework for multi-step AI pipelines, enabling the system to chain prompts, manage conversation memory, and invoke the OpenAI GPT API for script generation, topic research, and structural recommendations. This design ensures that complex AI workflows remain modular and maintainable. PostgreSQL is used as the primary persistent data store for user accounts, podcast projects, and generated content, while Redis provides in-memory caching for session tokens and frequently reused AI outputs, significantly reducing response latency. Authentication is enforced through JWT-based tokens combined with OAuth 2.0 for third-party integrations. The entire system is containerized via Docker and managed through a CI/CD pipeline powered by GitHub Actions, ensuring reliable, reproducible deployments.

3.2 Architecture Style / Pattern

PodCraft follows the Model-View-Controller (MVC) architectural pattern. This pattern divides the system into three interconnected components, enabling separation of concerns and independent development of each layer.

Architecture Pattern Diagram:



MVC Justification

- **Multiple Views:** PodCraft supports both web and mobile platforms (React JS for web, React Native for mobile). Both views share the same backend Controllers and Models, eliminating code duplication and ensuring consistent behavior across platforms. When a change is made to the business logic, both platforms inherit the update automatically.
- **Separation of Concerns:** The UI code (React components) handles only presentation logic - rendering screens, capturing user input, and displaying AI-generated results. The business logic (Node.js and AI Agents) handles data processing, API calls, and episode planning algorithms. This separation means a designer can modify the UI without understanding the AI pipeline, and an AI engineer can improve the agents without touching frontend code.
- **Testability:** Each MVC component can be tested in isolation. The View can be tested using Jest and React Testing Library without needing a live database. Controllers can be unit-tested with mock requests. Models can be tested independently using in-memory MongoDB. This isolation makes bugs easier to locate and fix.
- **Maintainability:** Because the View depends only on the Controller's API interface (not its internal implementation), developers can rewrite the AI agent logic entirely without changing a single line of UI code. Similarly, the database schema can evolve without breaking the frontend as long as the Controller's API contract remains stable.
- **Agent Integration:** Each of PodCraft's five AI agents (Intent Analyzer, Topic Research, Question Generator, Hook Suggester, Outline Planner) is implemented as a specialized Controller. They sit alongside the main Express controllers, handling specific AI tasks while following the same MVC conventions. This keeps the agent workflow organized and maintainable.

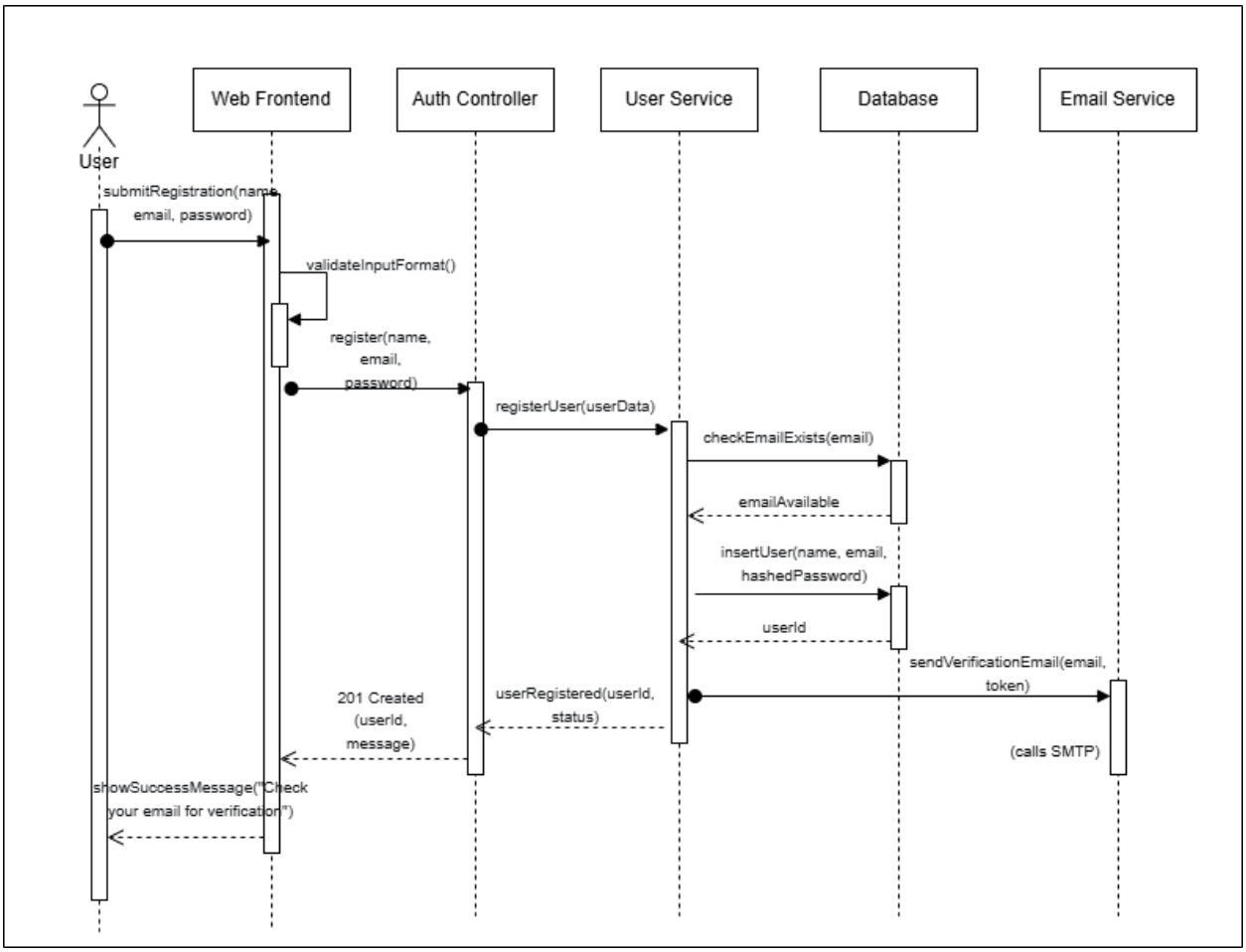
Component Responsibilities

- **View:** Displays UI, captures user input, and renders AI outputs. Technologies used: React JS, React Native, Tailwind CSS.
- **Controller:** Handles HTTP requests, validates input, orchestrates AI agents, and updates Models. Technologies used: Node.js, Express, 5 AI Agents.
- **Model:** Defines data structure, performs database operations, and enforces business rules. Technologies used: MongoDB, Mongoose ODM.

3.3 Design Models

3.3.1 Sequence Diagrams:

Module 1: UC1 User Registration:



Purpose:
Shows how a new user creates an account in PodCraft.

Key Elements:

- **Actor:** User
- **Lifelines:** WebFrontend, AuthController, UserService, Database, EmailService
- **Trigger:** User submits registration form
- **Outcome:** User account created, verification email sent

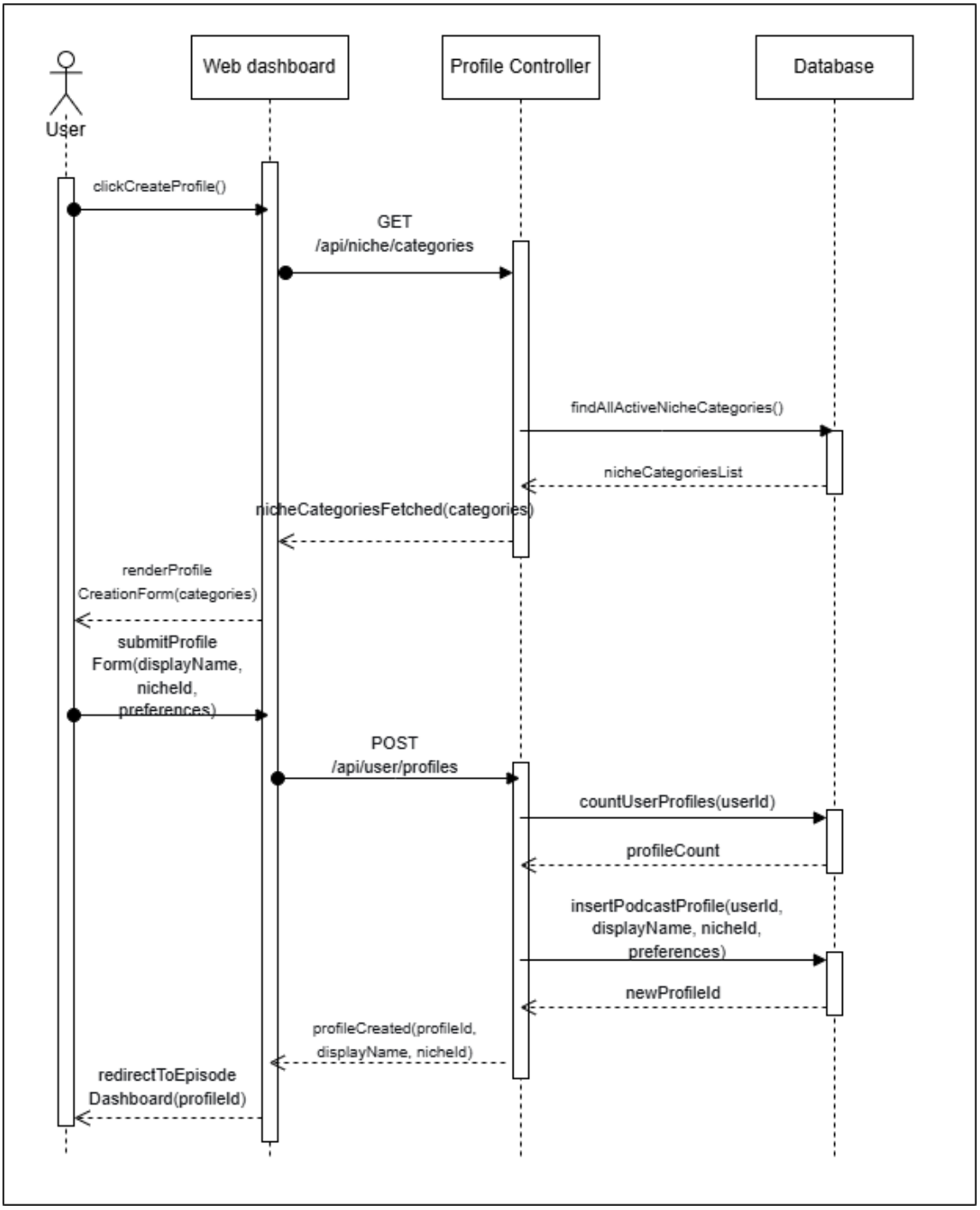
What Happens:

- User submits name, email, password

- WebFrontend validates input format
- AuthController receives request and calls UserService
- UserService checks if email already exists in Database
- If available, UserService inserts new user record
- UserService sends verification email (asynchronous)
- Success response returns to user

Key Design Decision: Email sending is **asynchronous** so user doesn't wait.

Module 1: UC4 Create Podcast Profile:



Purpose:
Shows how an authenticated user creates a podcast profile with niche selection.
Key Elements:

- **Actor:** Podcaster (authenticated user)

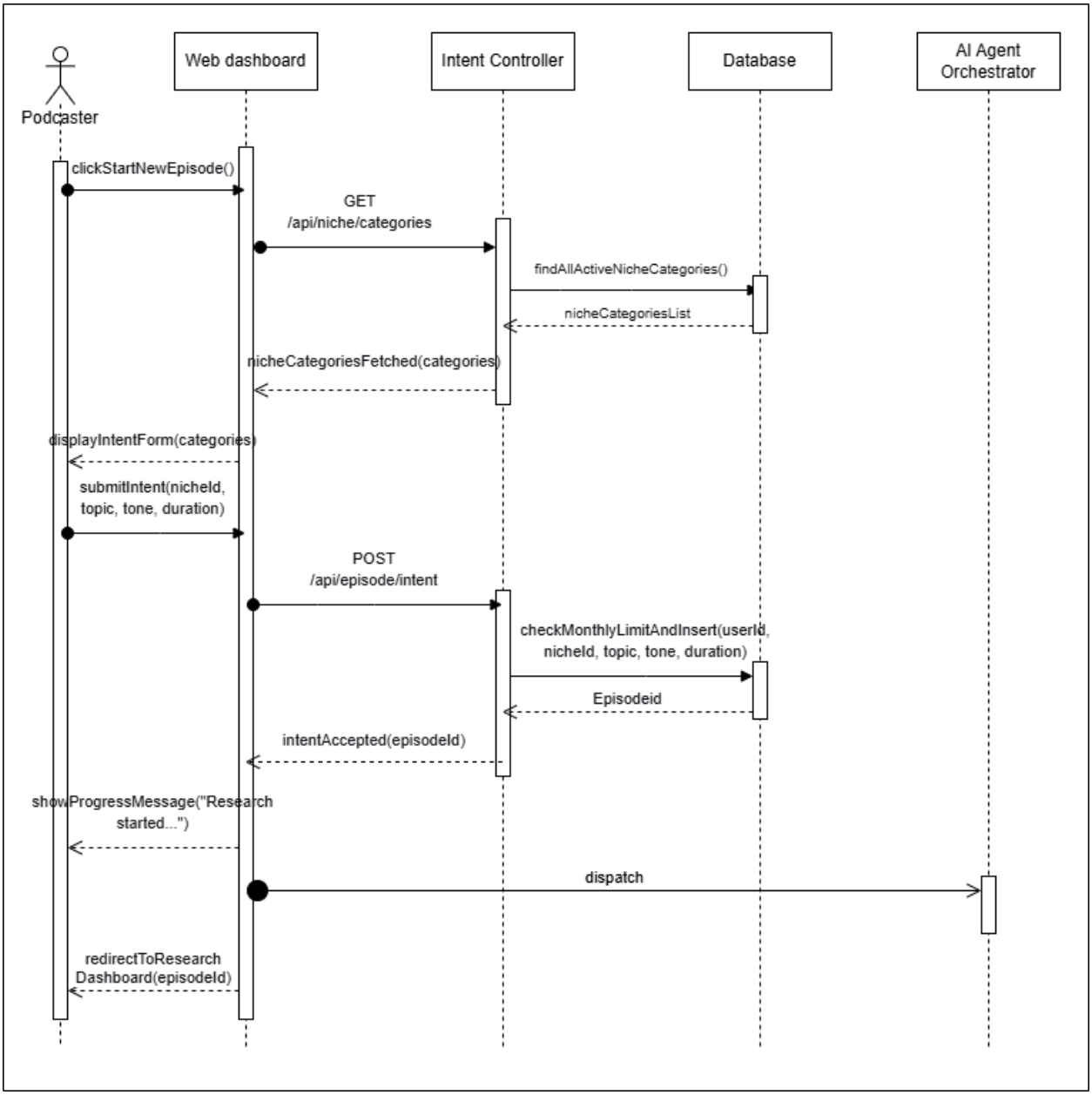
- **Lifelines:** WebDashboard, ProfileController, Database
- **Trigger:** Podcaster clicks "Create Profile"
- **Outcome:** New podcast profile saved, user redirected to dashboard

What Happens:

- Podcaster navigates to Create Profile screen
- WebDashboard fetches niche categories from Database
- User selects niche and enters display name
- ProfileController checks user's profile count (max 2 for free tier)
- If under limit, profile is inserted into Database
- User redirected to Episode Dashboard

Key Design Decision: Niche categories are cached to reduce database calls.

Module 2: UC1 Provide Podcast Niche and Topic:



Purpose:

Shows how a podcaster submits episode intent, triggering the AI agent pipeline.

Key Elements:

- **Actor:** Podcaster
- **Lifelines:** WebDashboard, IntentController, Database, AIAgentOrchestrator
- **Trigger:** Podcaster clicks "Analyze Intent"
- **Outcome:** Episode created, AI research pipeline triggered

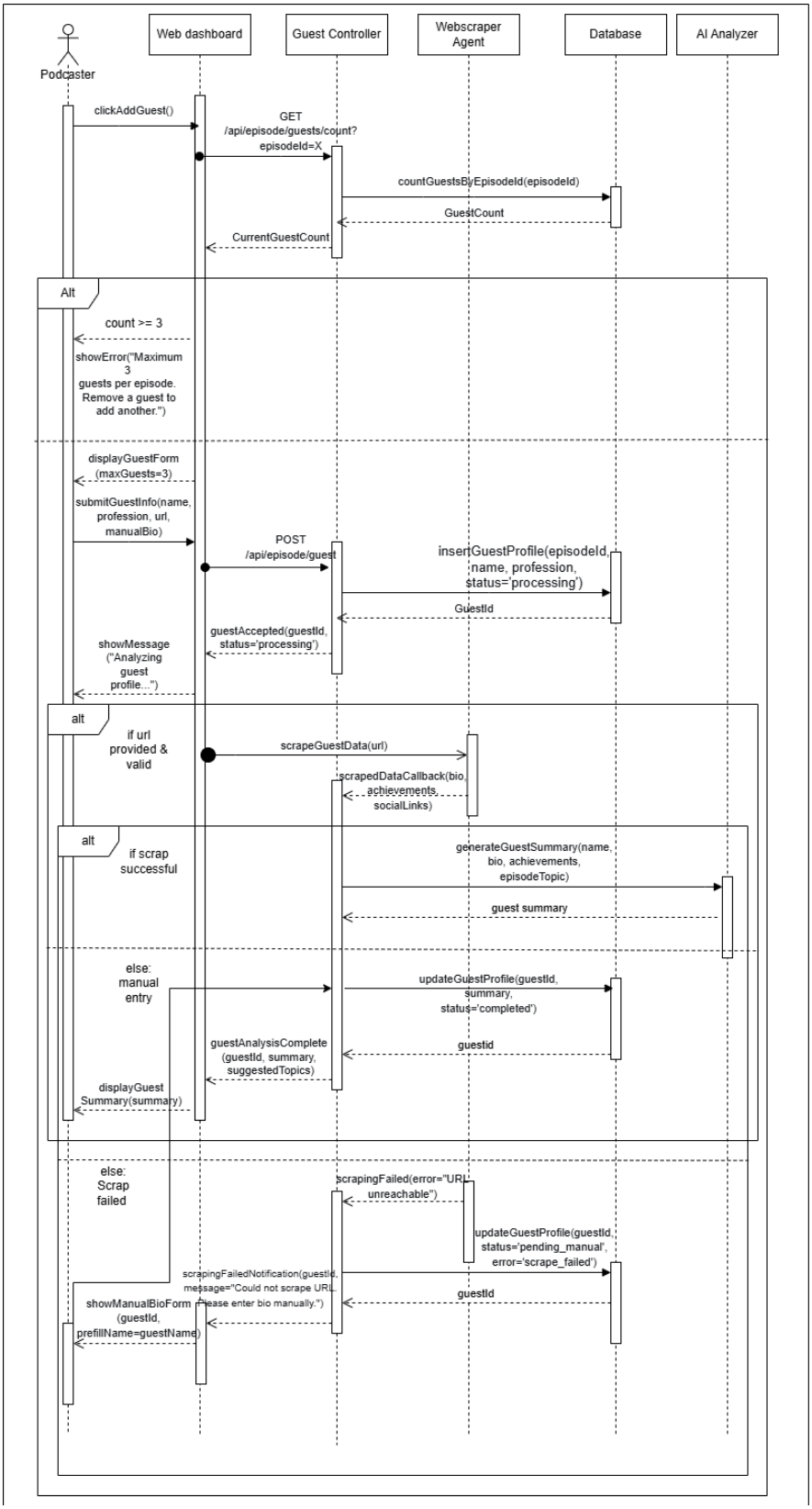
What Happens:

- Podcaster selects niche, enters topic, tone, and duration
- IntentController checks monthly episode limit (3 for free tier)

- Episode intent record inserted into Database
- User receives confirmation message
- AIAgentOrchestrator triggered asynchronously to start research
- User redirected to Research Dashboard

Key Design Decision: AI agent trigger is **asynchronous** - research happens in background.

Module 2: UC 2 Submit Guest Profile:



Purpose:
Shows how a podcaster adds a guest to an episode with URL scraping and AI analysis.
Key Elements:

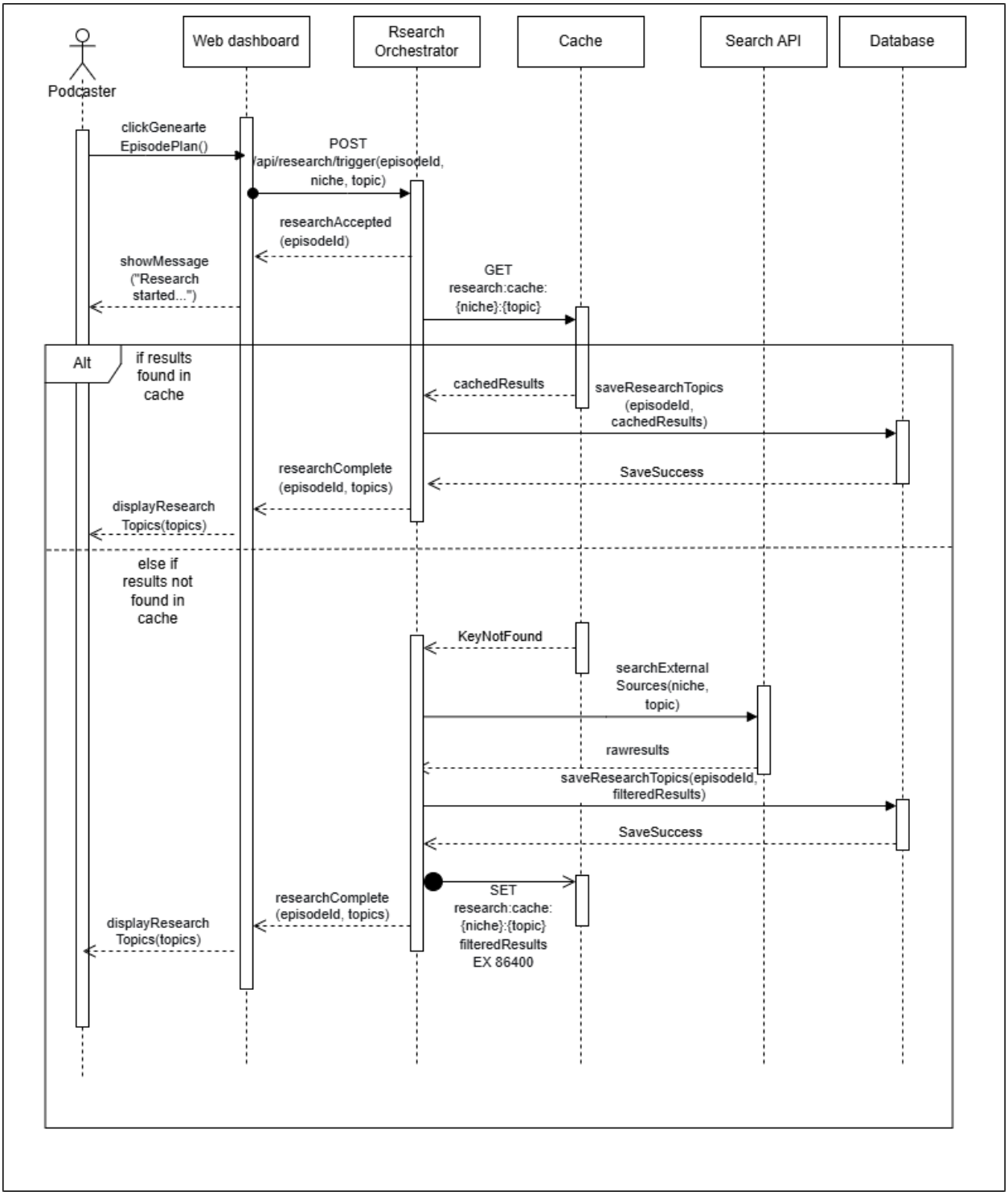
- **Actor:** Podcaster
- **Lifelines:** WebDashboard, GuestController, WebScraperAgent, Database
- **Trigger:** Podcaster submits guest name and URL
- **Outcome:** Guest profile analyzed and summarized

What Happens:

- Podcaster clicks "Add Guest" and fills form
- GuestController checks guest limit (max 3 per episode)
- Guest record created with status 'processing'
- If URL provided, WebScraperAgent scrapes guest data asynchronously
- AI generates summary from scraped data or manual bio
- Guest profile inserted into Database with all data at once
- Summary displayed to podcaster

Key Design Decision: Guest is inserted **only once** after all processing completes - no placeholder updates.

Module 3: UC1 Initiate Topic Research:



Purpose:
Shows how the system researches topics based on episode niche and topic.
Key Elements:

- **Actor:** Podcaster (manual trigger) or System (auto)

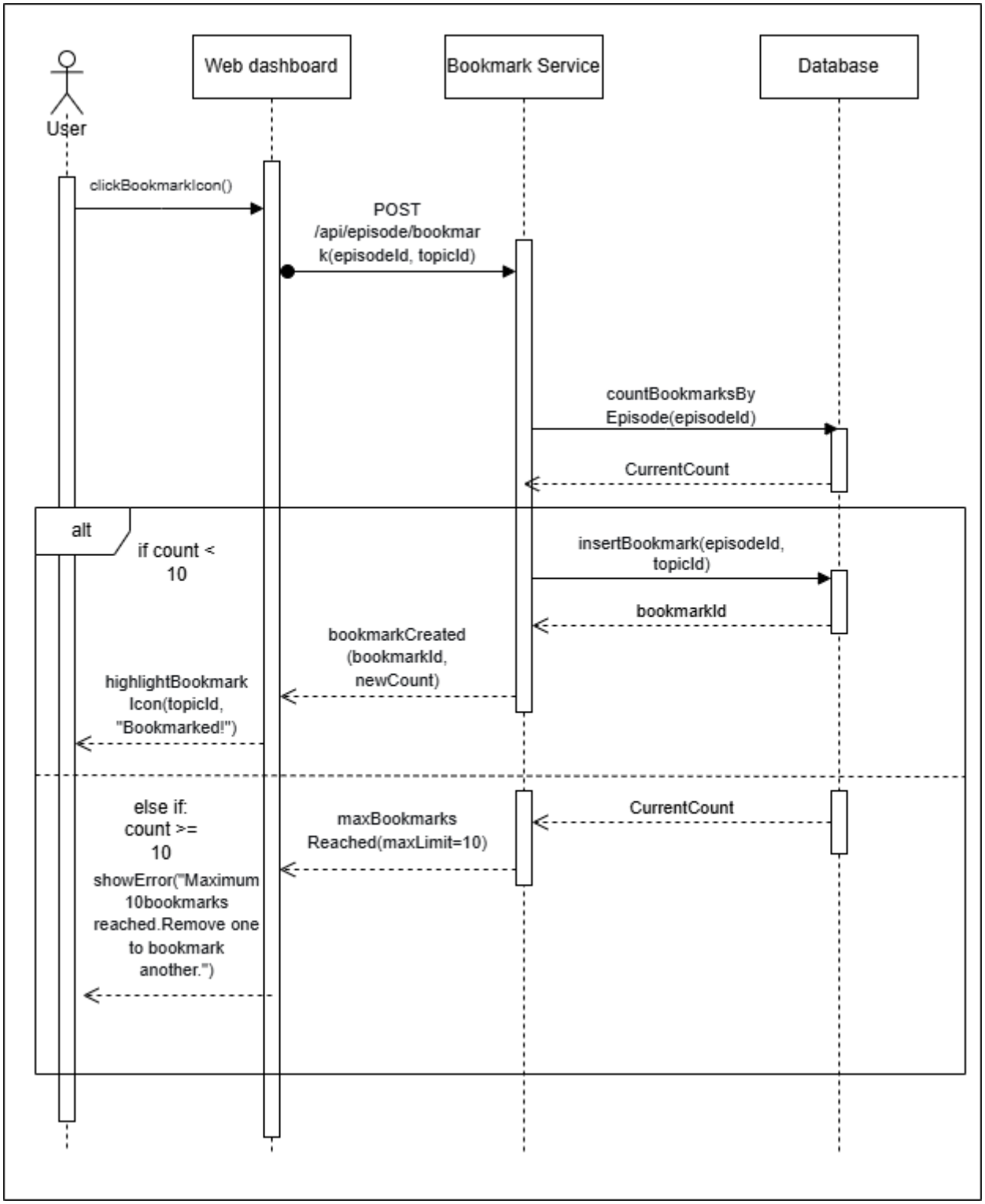
- **Lifelines:** WebDashboard, ResearchOrchestrator, Cache, SearchAPI, Database
- **Trigger:** Auto after episode creation OR Podcaster clicks "Run Research"
- **Outcome:** Research topics saved and displayed

What Happens:

- Research triggered (auto or manual)
- ResearchOrchestrator checks Cache for existing results
- If cache hit → use cached results (skip API calls)
- If cache miss → SearchAPI called for external sources
- Results filtered by relevance threshold (0.65)
- Topics saved to Database
- Results cached for 24 hours (for future requests)
- Topics displayed to podcaster

Key Design Decision: Cache prevents redundant API calls for same niche-topic pairs.

Module 3: UC4 Bookmark Preferred Topics:



Purpose:
Shows how a podcaster bookmarks relevant topics for question generation.

Key Elements:

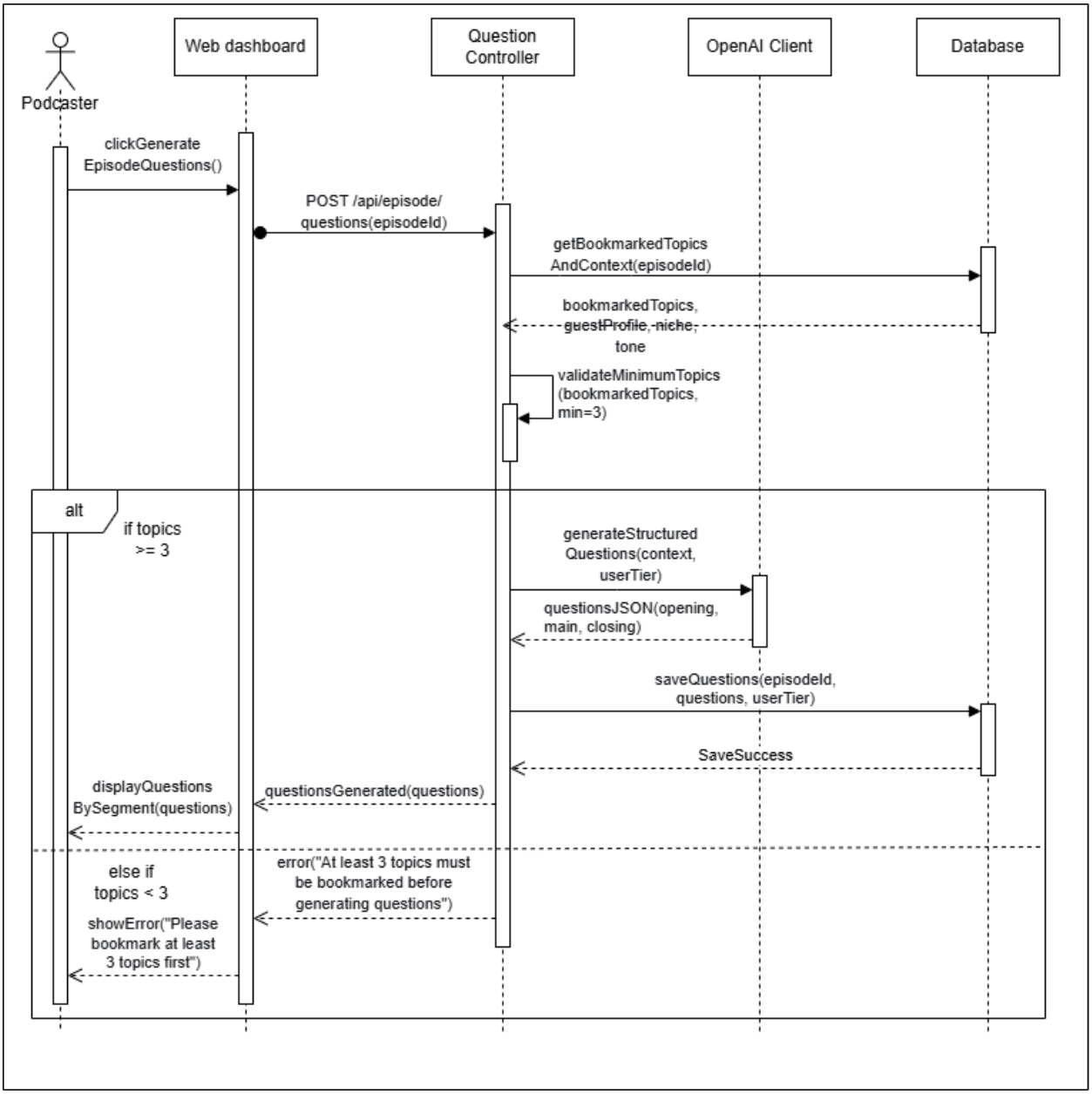
- **Actor:** Podcaster
- **Lifelines:** WebDashboard, BookmarkService, Database
- **Trigger:** Podcaster clicks bookmark icon
- **Outcome:** Topic bookmarked, count updated

What Happens:

- Podcaster clicks bookmark icon on a topic
- BookmarkService checks current bookmark count (max 10)
- If under limit, bookmark inserted into Database
- UI highlights the bookmarked topic
- New count displayed to user

Key Design Decision: Bookmarking is **synchronous** for immediate visual feedback.
Unbookmarking is a separate use case (M2-UC5: Reject Irrelevant Topics) - not included here.

Module4: UC1 Generate Episode Questions



Purpose:
Shows how the system generates structured interview questions based on bookmarked topics.
Key Elements:

- **Actor:** Podcaster
- **Lifelines:** WebDashboard, QuestionController, Database, OpenAIClient
- **Trigger:** Podcaster clicks "Generate Questions"
- **Outcome:** 10-20 questions organized by segment

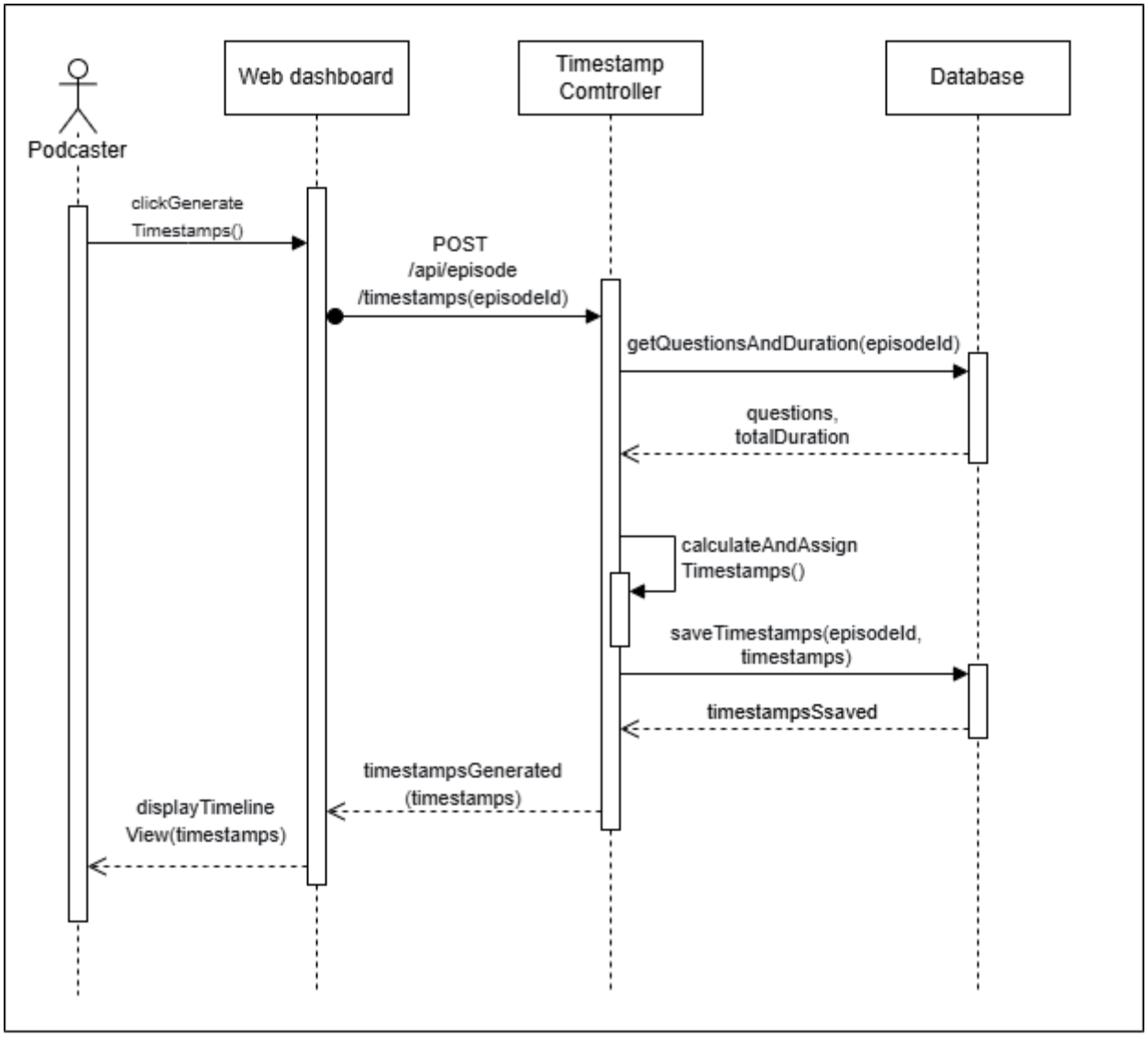
What Happens:

- Podcaster clicks "Generate Questions"
- QuestionController retrieves bookmarked topics and context from Database

- System validates at least 3 topics are bookmarked
- OpenAIClient generates questions organized into Opening, Main, Closing segments
- Questions saved to Database
- Questions displayed to podcaster grouped by segment

Key Design Decision: Free-tier users receive only first 8 questions (limited by BR-10).

Module 4: UC6 Auto-Generate Timestamp Structure:



Purpose:
Shows how the system automatically allocates time to each question based on episode duration.
Key Elements:

- **Actor:** Podcaster
- **Lifelines:** WebDashboard, TimestampController, Database
- **Trigger:** Podcaster clicks "Generate Timestamps"
- **Outcome:** Timestamps assigned to each question

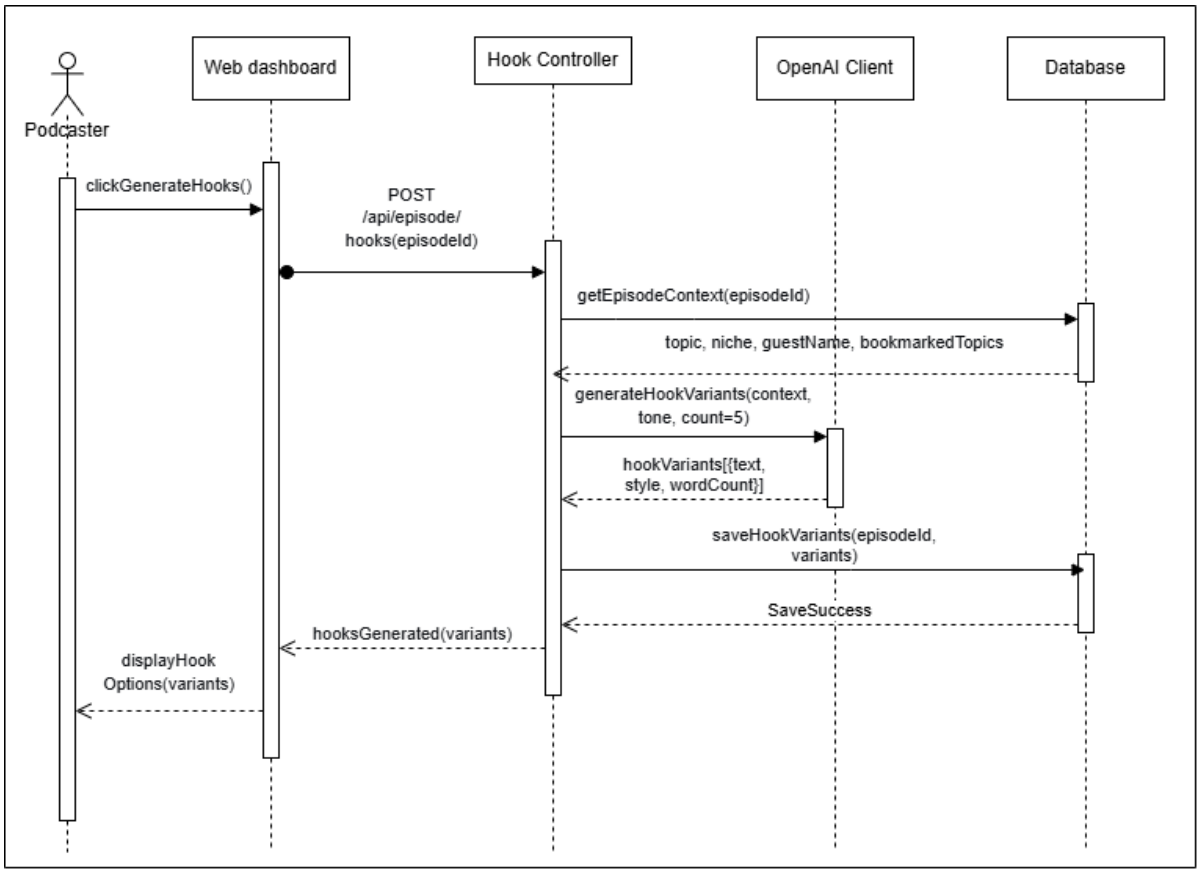
What Happens:

- Podcaster clicks "Generate Timestamps"
- TimestampController retrieves questions and episode duration from Database
- System calculates time allocation:
- Reserves 2-3 min for intro
- Reserves 3-5 min for outro

- Distributes remaining time across main questions
- Timestamps saved to Database
- Timeline view displayed to podcaster

Key Design Decision: Calculation is **synchronous** and fast - no external API calls needed.

Module 5: UC1 Generate Multiple Hook Variants:



Purpose:

Shows how the system generates multiple hook options for episode introduction.

Key Elements:

- **Actor:** Podcaster
- **Lifelines:** WebDashboard, HookController, Database, OpenAIClient
- **Trigger:** Podcaster selects tone and clicks "Generate Hooks"
- **Outcome:** 3-5 hook variants displayed with style tags

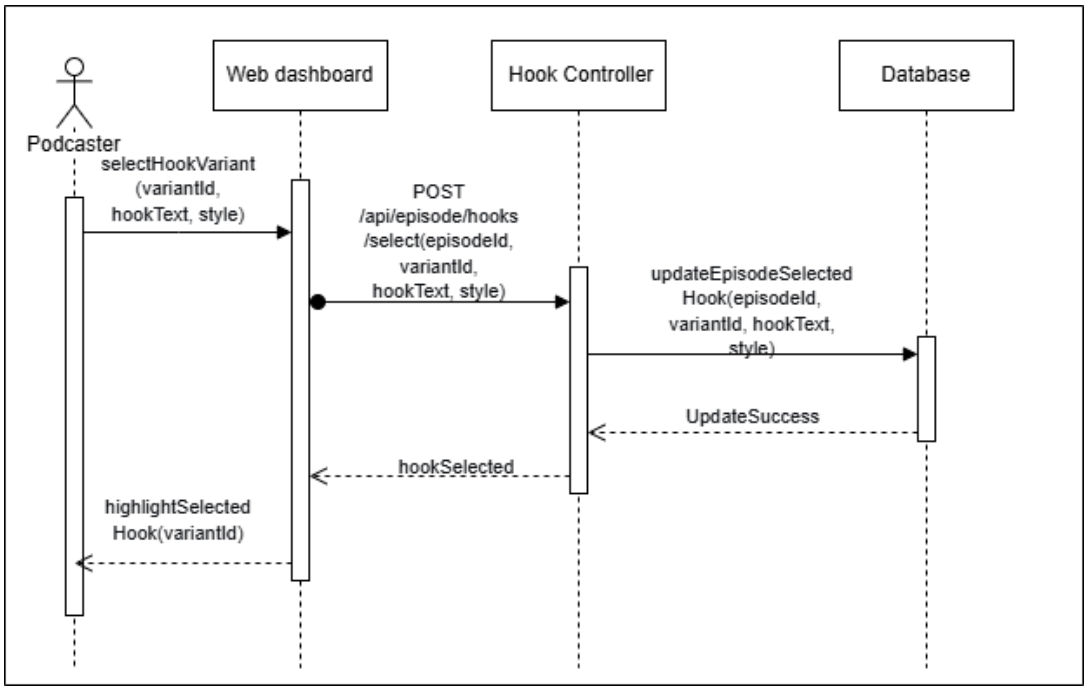
What Happens:

- Podcaster selects tone (Dramatic, Curious, Humorous, Inspirational)
- Podcaster clicks "Generate Hooks"
- HookController retrieves episode context from Database
- OpenAIClient generates 3-5 hook variants (30-120 words each)
- Each hook tagged with rhetorical style
- Variants saved to Database

- Hooks displayed to podcaster as selectable cards

Key Design Decision: Multiple variants give podcaster creative choice while maintaining consistent tone.

Module 5: UC2 Select Preferred Hook Style:



Purpose:
Shows how a podcaster selects and saves a preferred hook variant.

Key Elements:

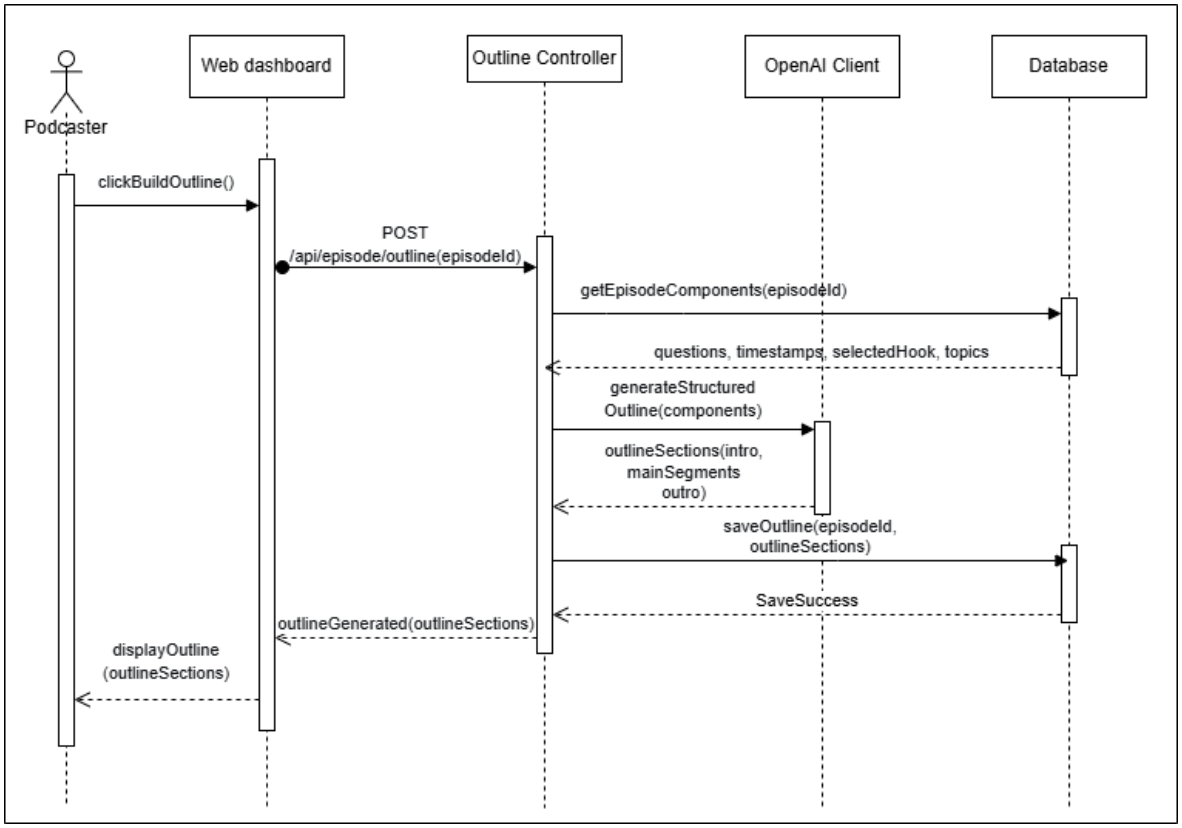
- **Actor:** Podcaster
- **Lifelines:** WebDashboard, HookController, Database
- **Trigger:** Podcaster taps "Select" on a hook variant
- **Outcome:** Selected hook saved to episode

What Happens:

- Podcaster selects a hook variant (UI already has the text and style)
- HookController receives variantId, text, and style
- Database updates episode with selected hook
- UI highlights the selected hook
- Confirmation shown to user

Key Design Decision: No database GET needed - UI already has hook data. Only UPDATE occurs.

Module 6: UC1 Generate Full Episode Outline:



Purpose:
Shows how the system synthesizes all prior outputs into a complete episode outline.
Key Elements:

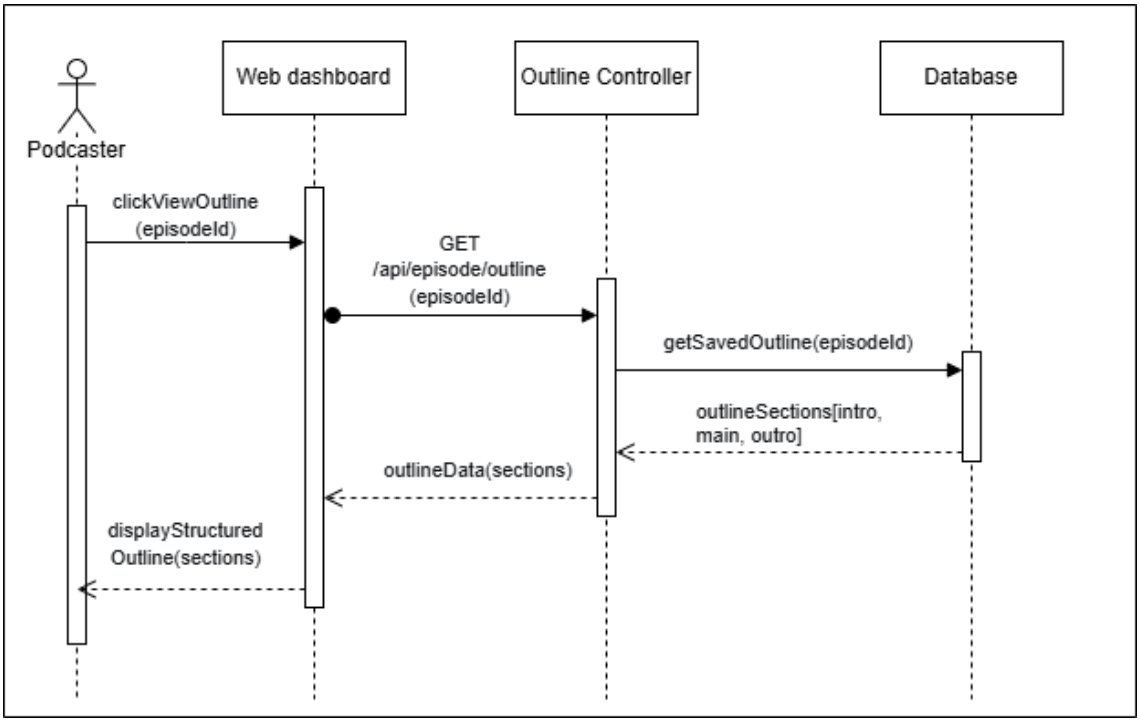
- **Actor:** Podcaster
- **Lifelines:** WebDashboard, OutlineController, Database, OpenAIClient
- **Trigger:** Podcaster clicks "Build Outline"
- **Outcome:** Complete episode outline with sections

What Happens:

- Podcaster clicks "Build Outline"
- OutlineController retrieves all episode components from Database:
- Questions
- Timestamps
- Selected hook
- Bookmarked topics
- OpenAIClient generates structured outline (Intro → Main Segments → Outro)
- Outline saved to Database
- Structured outline displayed to podcaster

Key Design Decision: One database call retrieves all components - no complex orchestration.

Module 6: UC2 View Outline in Structured Format:



Purpose:
Shows how a podcaster views a saved episode outline.
Key Elements:

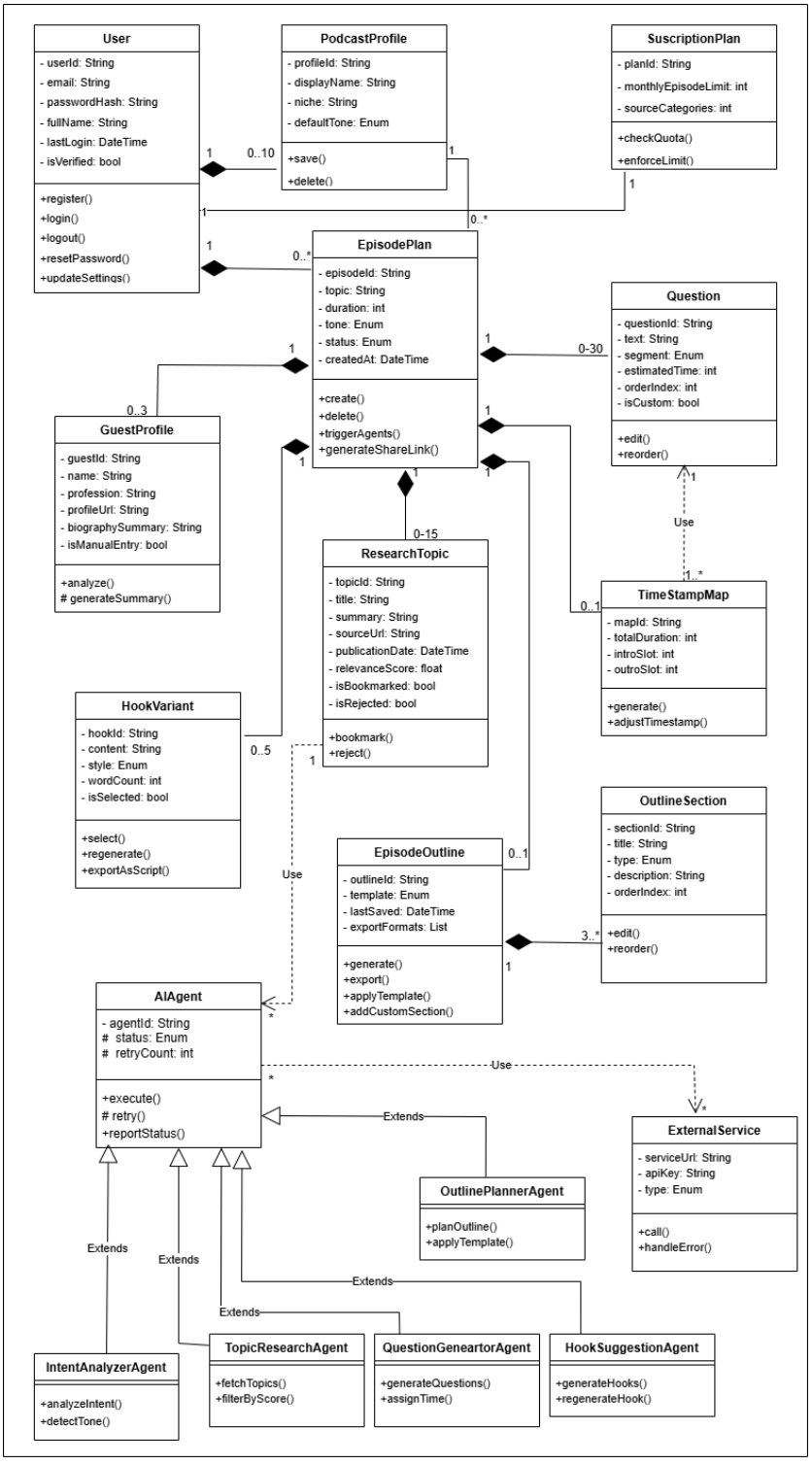
- **Actor:** Podcaster
- **Lifelines:** WebDashboard, OutlineController, Database
- **Trigger:** Podcaster clicks on saved episode
- **Outcome:** Outline displayed in readable format

What Happens:

- Podcaster clicks "View Outline" on an episode
- OutlineController retrieves saved outline from Database
- Outline sections (intro, main segments, outro) returned
- WebDashboard formats and displays outline in structured view

Key Design Decision: This is a simple read operation - no processing or generation involved.

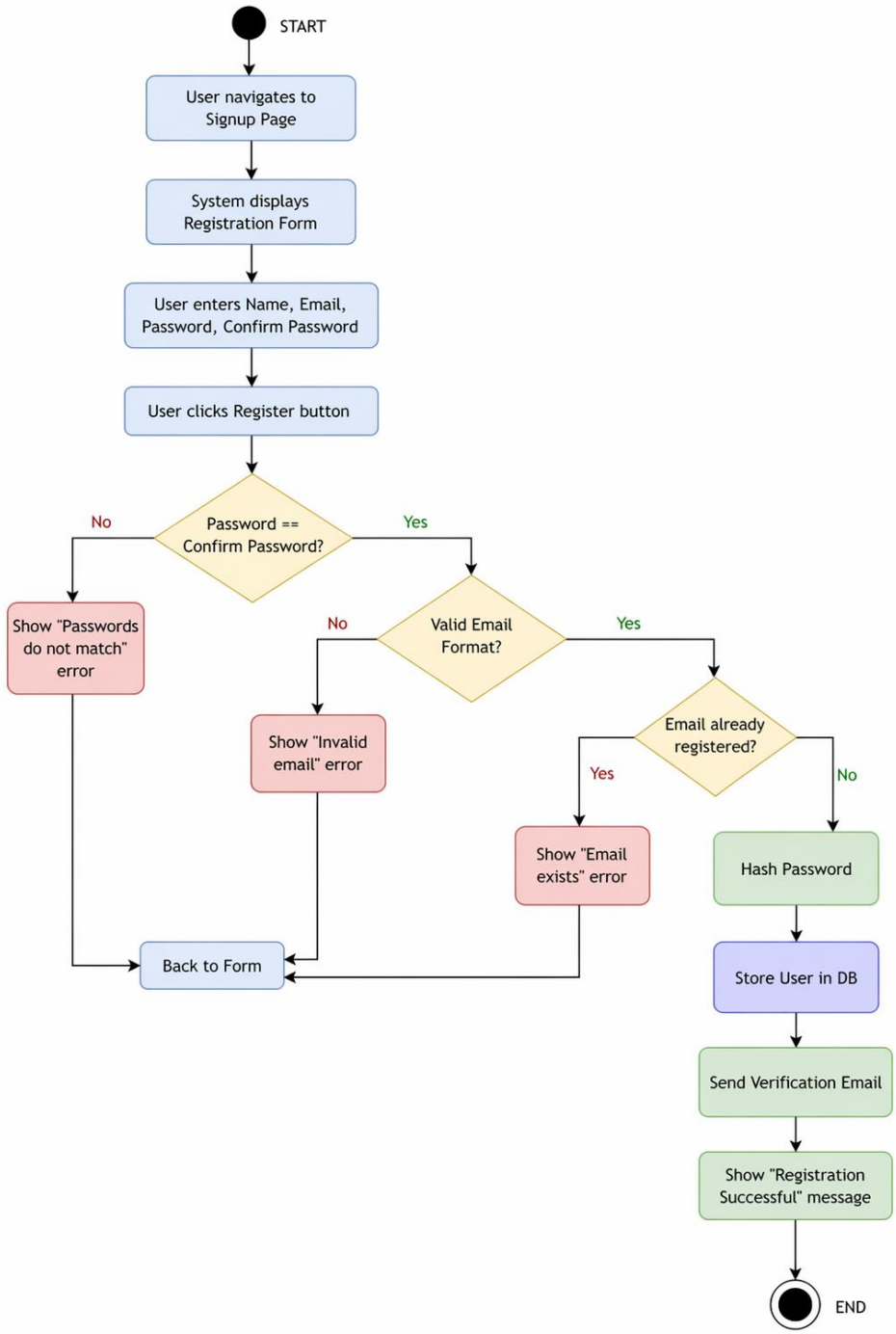
3.3.2 Class Diagram:



The class diagram illustrates the static structure of PodCraft following MVC conventions. Controller classes (Agents) are shown in orange, Model classes (Data Entities) in blue. View classes are omitted as they represent UI components.

3.3.3 Activity Diagrams:

Module 1: UC1 User Registration:



Overview

- Models the complete user registration workflow in PodCraft's authentication module.
- Covers every decision point from signup page to verified account creation.

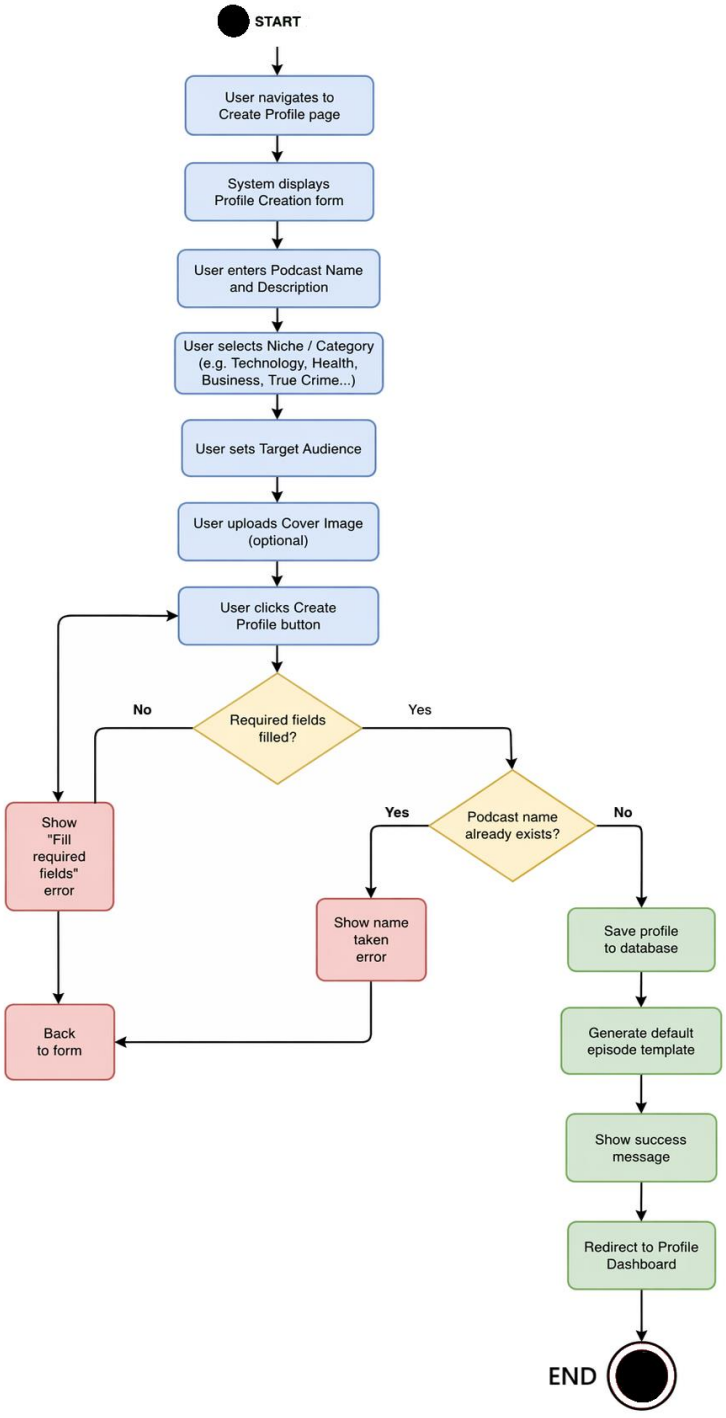
Key Elements

- User navigates to signup page → system renders registration form.
- User enters name, email, password, and confirmation → clicks Register.
- Validation 1: Passwords match? If not → show error → redirect to form.
- Validation 2: Email format valid? If not → show error → redirect to form.
- Validation 3: Email already registered? If yes → show error → redirect to form.
- All checks pass → hash password (bcrypt) → persist user record → send verification email → show "Registration Successful" → end state.

Contribution

- User authentication is the gateway to all PodCraft functionality.
- Ensures robust client-side and server-side validation.
- Prevents duplicate accounts and protects password integrity.

Module 1: UC4 Create Podcast Profile:



Overview

- Represents the workflow for an authenticated user to create a podcast profile.
- The profile serves as the container for all subsequent episode planning.

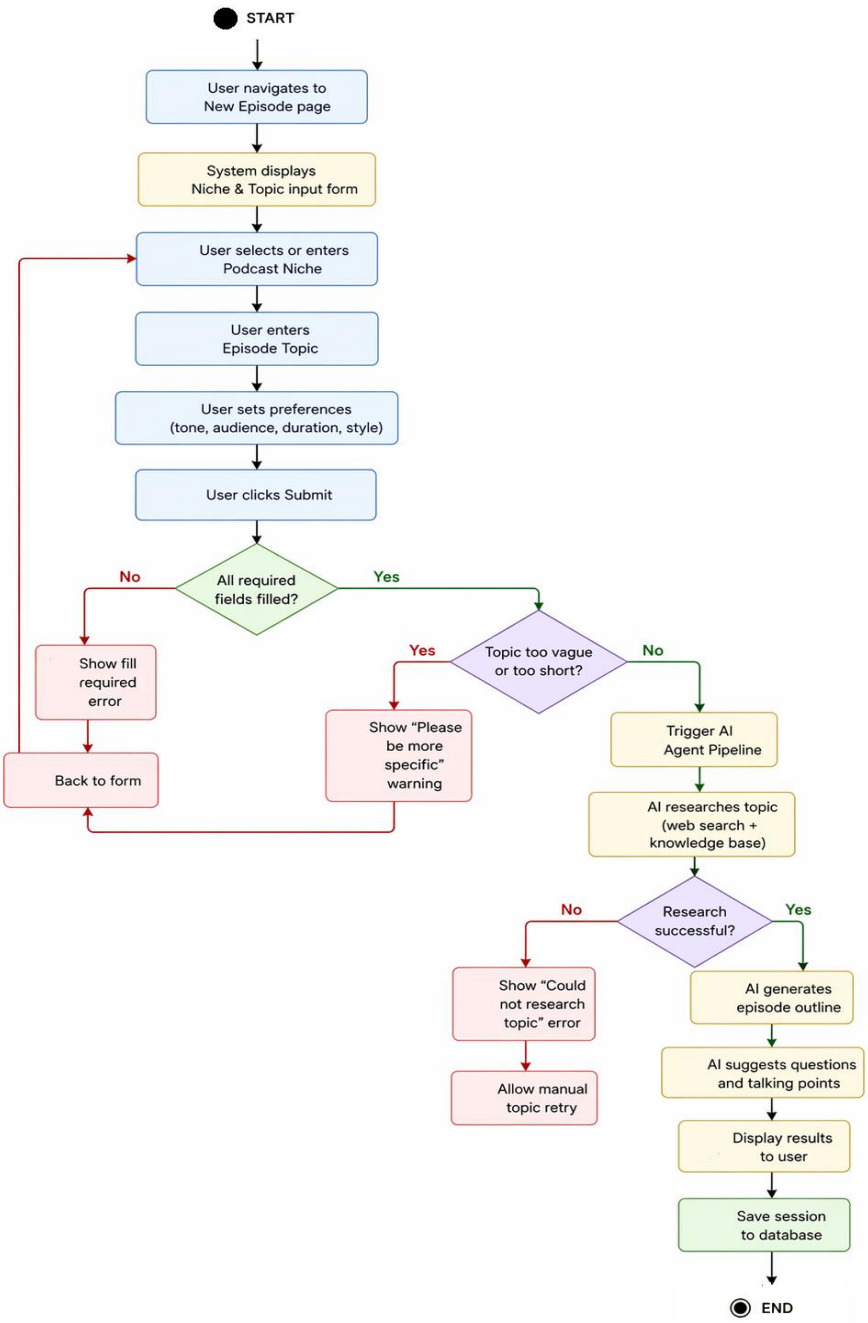
Key Elements

- User navigates to Create Profile page → views profile creation form.
- User supplies: podcast name, description, niche, target audience, optional cover image.
- Clicks Create Profile → system checks required fields. If incomplete → error → redisplay form.
- System checks podcast name uniqueness. If name exists → error → redirect to form.
- Both checks pass → save profile to database → auto-generate default episode template → show success message → redirect to Profile Dashboard.

Contribution

- Podcast profile is the core organisational entity for AI-generated content.
- Enforces uniqueness constraint on podcast names to prevent data integrity issues.
- Auto-generation of default template demonstrates agentic intent and reduces manual setup.

Module 2: UC1 Provide Podcast Niche and Topic:



Overview

- Models the primary input workflow that triggers PodCraft's agentic AI pipeline.
- Captures steps for providing niche and episode topic for autonomous agents.

Key Elements

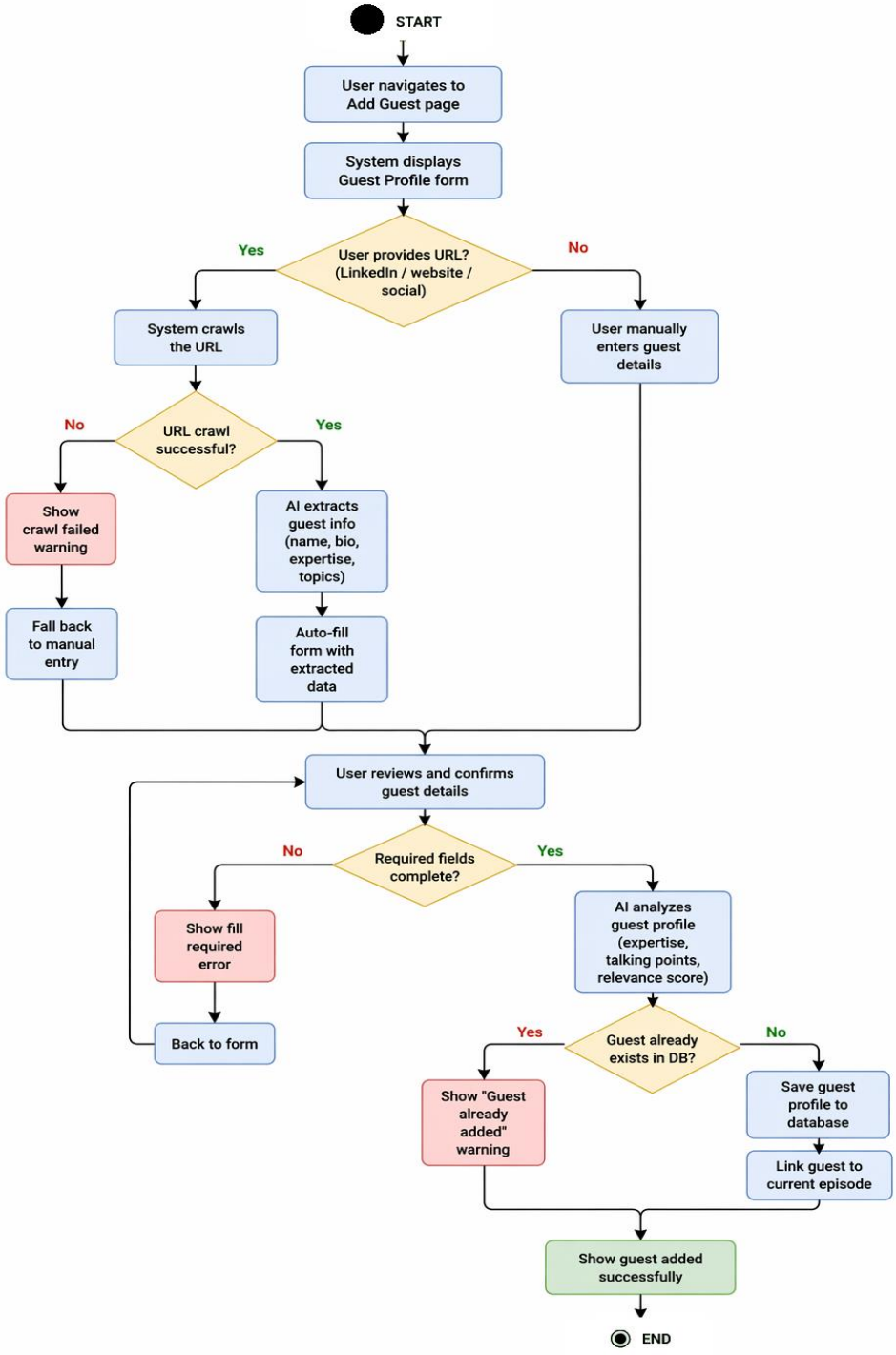
- User navigates to New Episode page → shown Niche & Topic input form.

- User selects/enters niche, types episode topic, configures tone, audience, duration, style → clicks Submit.
- Completeness check: missing fields → "Fill required fields" error.
- Topic specificity check: too vague/short → "Please be more specific" warning → return to form.
- Topic adequate → trigger AI Agent Pipeline → research agent queries web + internal KB.
- Research fails → "Could not research topic" error → manual retry option.
- Success → generate episode outline + suggested questions → display results → save session to database.

Contribution

- Entry point to the entire agentic workflow — most important diagram.
- Quality gate on topic specificity prevents shallow AI-generated content.
- Shows human-in-the-loop transitioning control to autonomous processing.

Module 2: UC 2 Submit Guest Profile:



Overview

- Captures dual-path workflow for adding a guest profile to an episode.
- Covers automated URL-scraping path and manual entry path.

Key Elements

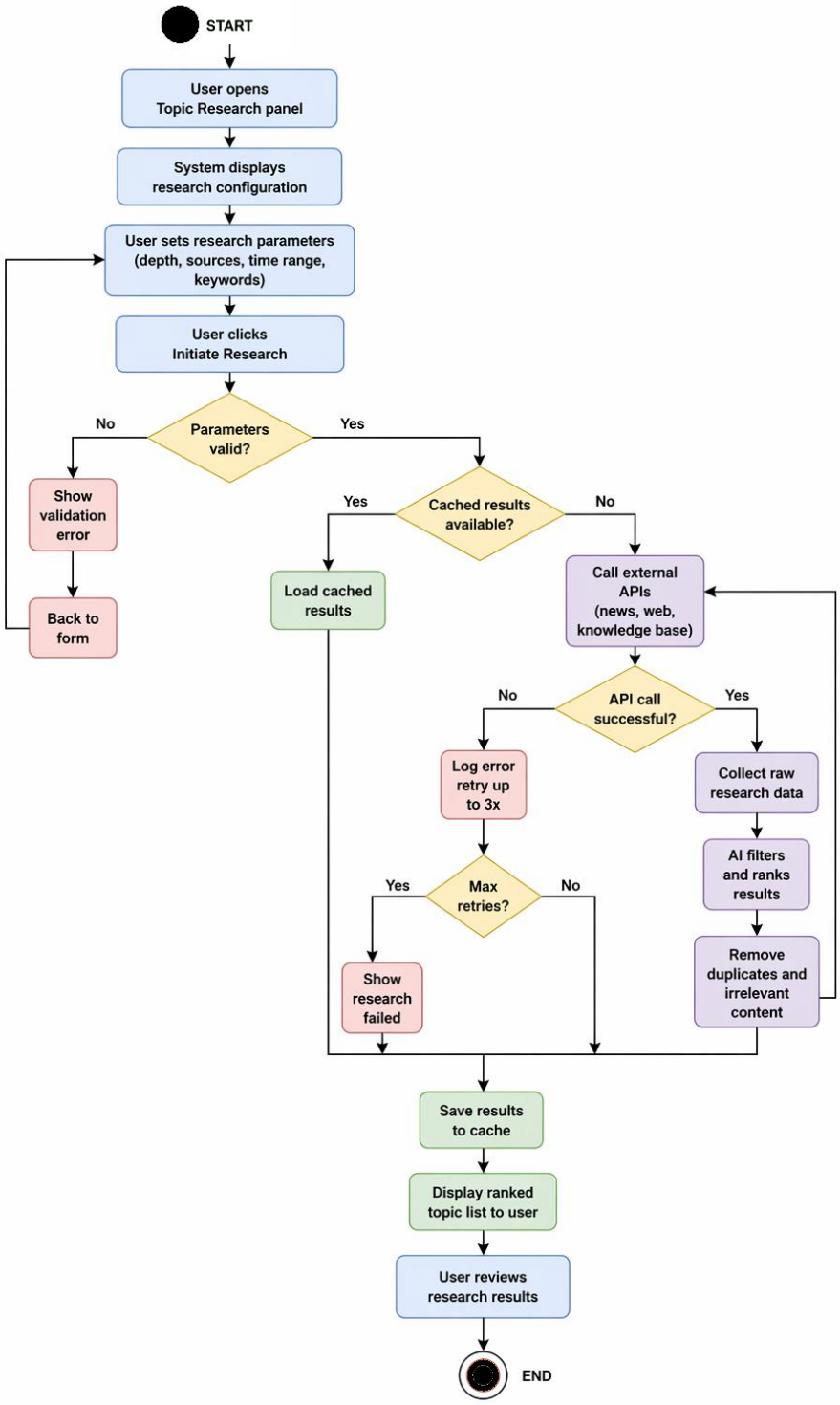
- User navigates to Add Guest page → receives Guest Profile form.
- Decision: URL provided? If yes → system crawls URL.
- Crawl fails → warning → fallback to manual entry.
- Crawl succeeds → AI extracts name, bio, expertise, topics → auto-fills form.
- User reviews and confirms guest details → completeness check. If incomplete → return to form.
- AI analyses guest profile → produces expertise scores, talking points, relevance rating.
- Duplicate guest check → if exists → "Guest already added" warning.
- New guest saved to database → linked to current episode → success confirmation.

Contribution

- Guest research is time-consuming for podcasters; this reduces preparation effort.
- Automated URL-scraping branch with manual fallback maintains data quality.

AI-driven profile analysis adds value beyond simple data entry.

Module 3: UC1 Initiate Topic Research:



Overview

- Models the behaviour of the Topic Research Agent (one of five core autonomous agents).
- Describes how the system responds to configurable research tasks against external and internal data sources.

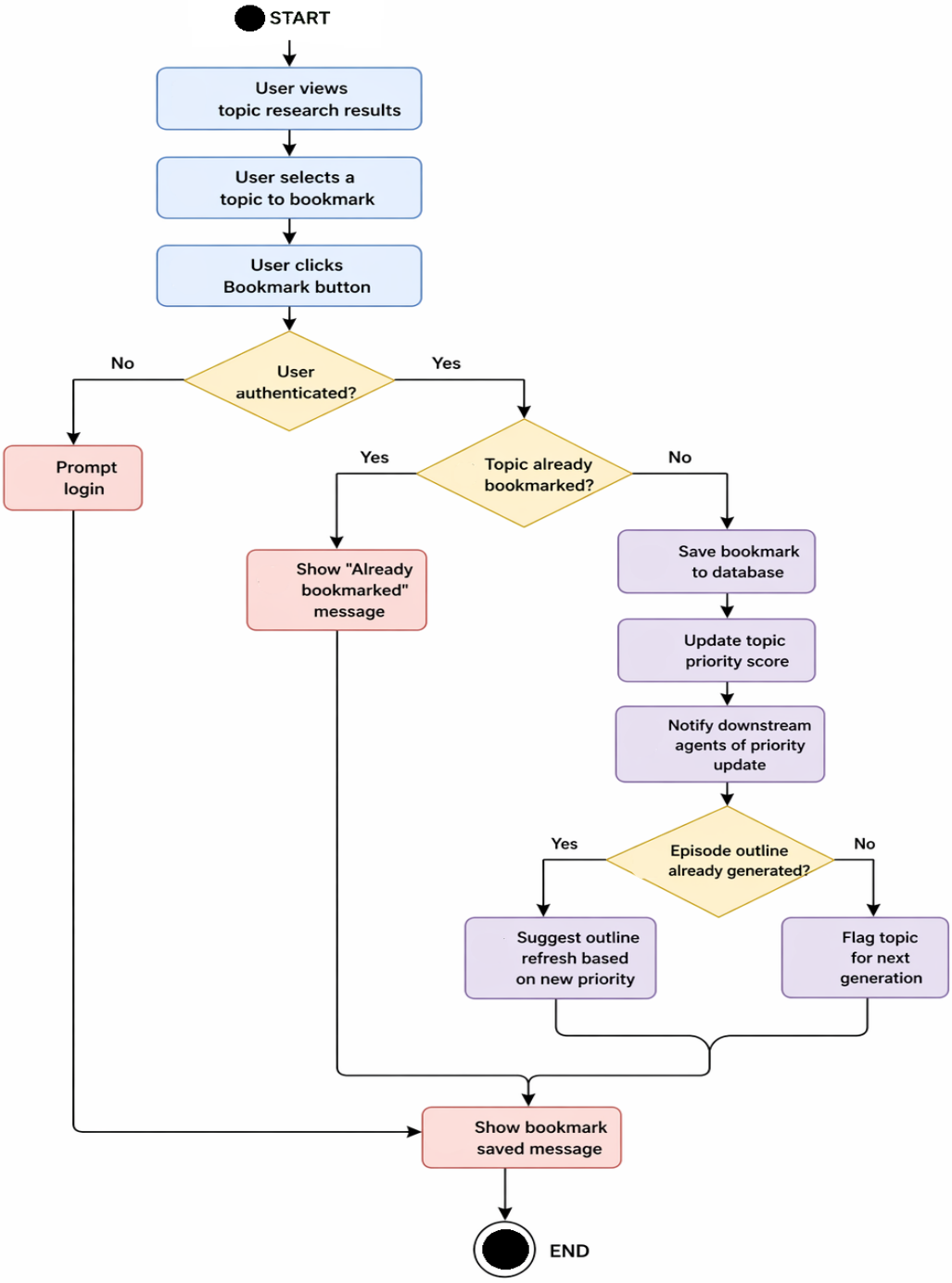
Key Elements

- User opens Topic Research panel → system displays research configuration interface.
- User sets: research depth, preferred sources, time range, keywords → clicks Initiate Research.
- Parameter validation: invalid config → error → return to form.
- Check cached results (24-hour TTL). If cached → load immediately.
- No cache → call external APIs (news, web search, knowledge base).
- API call fails → logged and retried up to 3 times. Max retries reached → "Research failed" message.
- Successful API call → collect raw data → AI filters and ranks results → remove duplicates/irrelevant content.
- Refined results saved to cache → display ranked topic list → user reviews findings.

Contribution

- Exposes caching and retry logic that affects system performance and reliability.
- Retry-with-limit pattern prevents cascading failures against external APIs.
- Cache-check branch meets responsiveness requirements by avoiding redundant external calls.

Module 3: UC4 Bookmark Preferred Topics:



Overview

- Models the topic bookmarking workflow for flagging high-priority research topics.
- Shows how a save action triggers a chain of intelligent agentic updates.

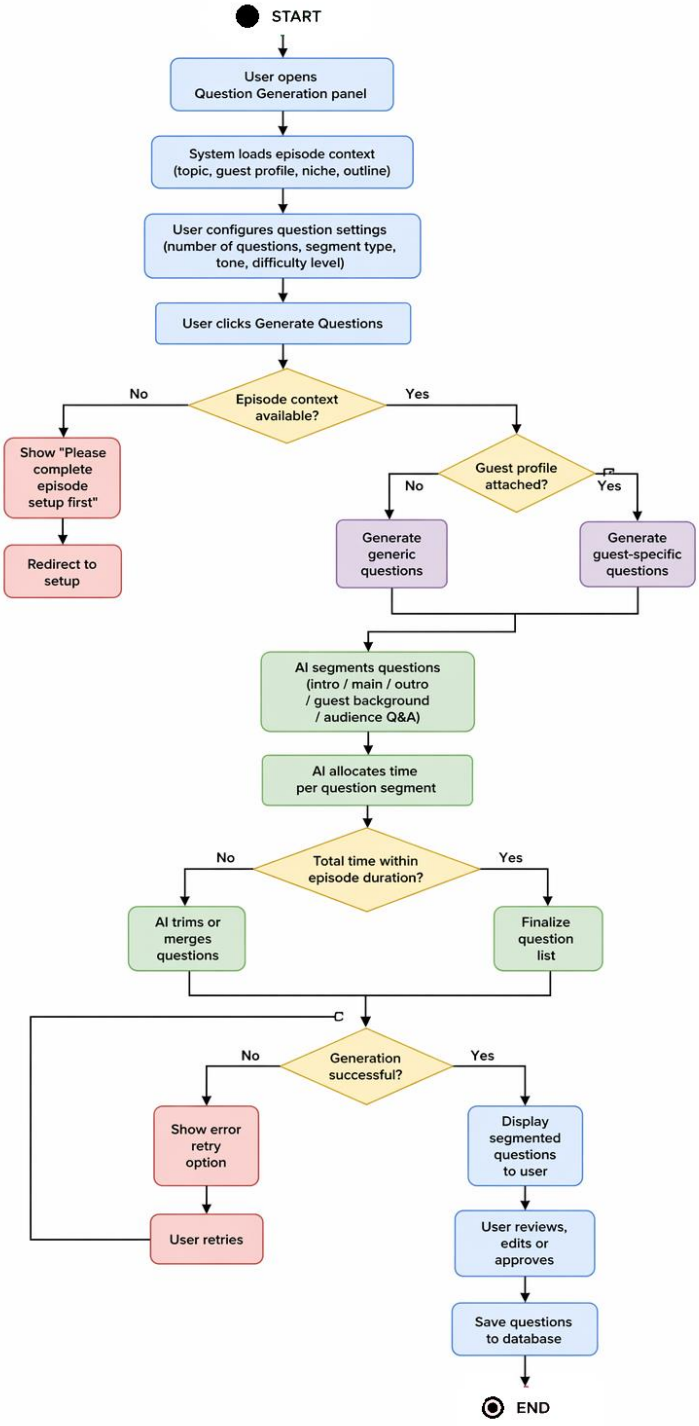
Key Elements

- User selects a topic from research results view → clicks Bookmark.
- System verifies authentication: unauthenticated → prompt to log in.
- Check if topic already bookmarked → if yes → "Already bookmarked" message.
- New bookmark → save to database → update topic's priority score → notify downstream agents of priority change.
- Check if episode outline already generated for this topic.
- If outline exists → suggest refreshing outline to incorporate updated priority.
- If no outline yet → flag topic for prioritised consideration in next outline generation.
- User receives "Bookmark saved" confirmation.

Contribution

- Illustrates agentic, reactive nature of PodCraft.
- User action at research stage propagates state changes forward to outline generation.
- Inter-agent notification pattern maintains coherence across autonomous pipeline.

Module4: UC1 Generate Episode Questions



Overview

- Models the Question Generator Agent's workflow.
- Produces contextually appropriate, time-allocated interview questions tailored to episode topic, niche, and guest profile.

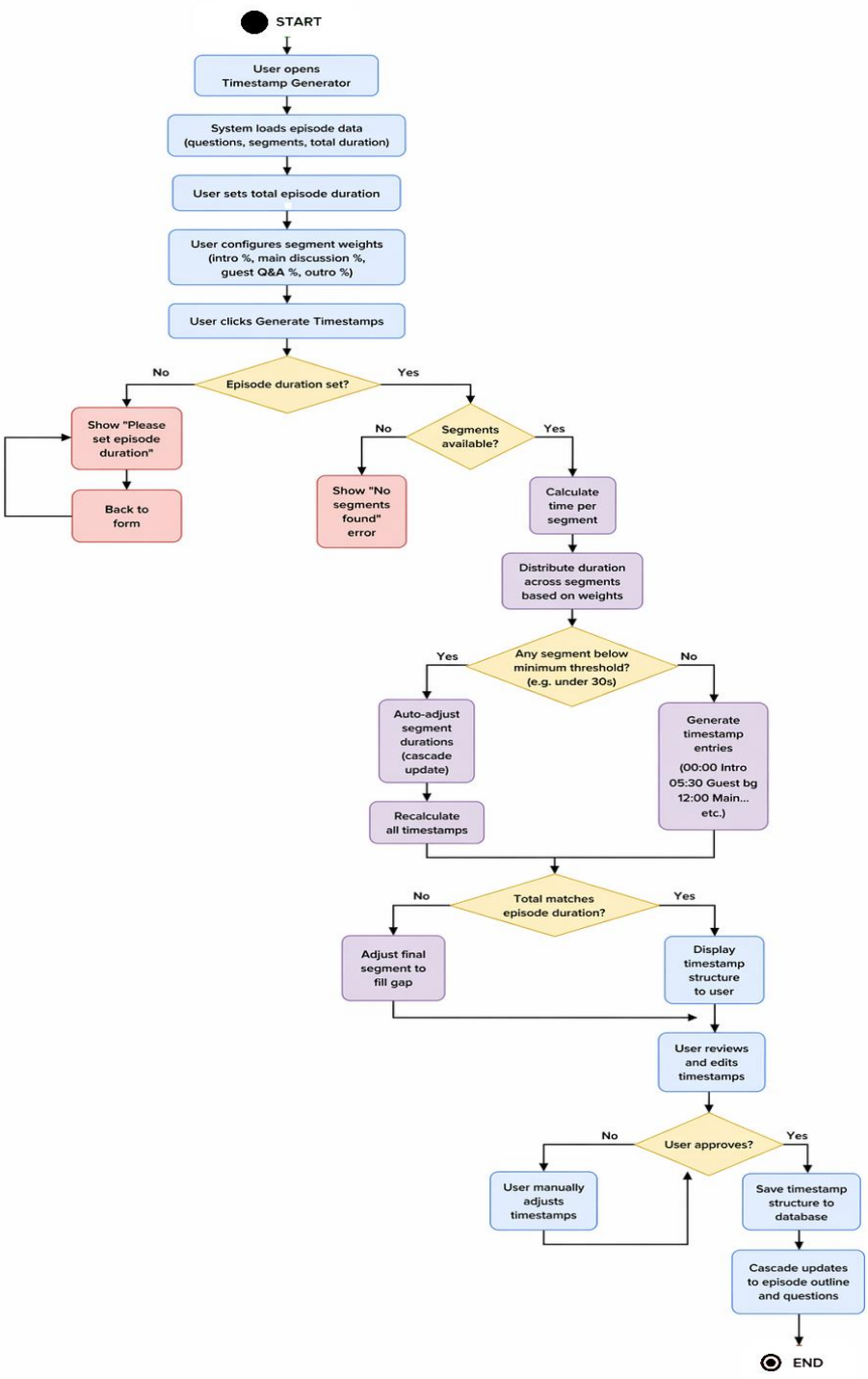
Key Elements

- User opens Question Generation panel → system loads episode context (topic, guest profile, niche, outline).
- User configures: number of questions, segment type distribution, tone, difficulty → clicks Generate Questions.
- Episode context missing → "Please complete episode setup first" error → redirect to setup.
- Context available → branch on guest profile attached.
- No guest profile → generic questions.
- Guest profile attached → guest-specific question generation.
- AI segments questions across: intro, main discussion, guest background, audience Q&A → allocates time per segment.
- Total time exceeds episode duration → AI trims or merges questions.
- Otherwise question list finalised.
- Generation success check. Failure → error + retry option.
- Success → display segmented questions → user reviews, edits, or approves → save to database.

Contribution

- Dual-path generation (generic vs guest-specific) demonstrates context-awareness.
- Time allocation and trim/merge mechanism enforces production constraints automatically.
- Relieves user from manual pacing calculations.

Module 4: UC6 Auto-Generate Timestamp Structure:



Overview

- Models the Timestamp Generation workflow.
- System automatically distributes episode duration across defined content segments and produces broadcast-ready timestamp structure.

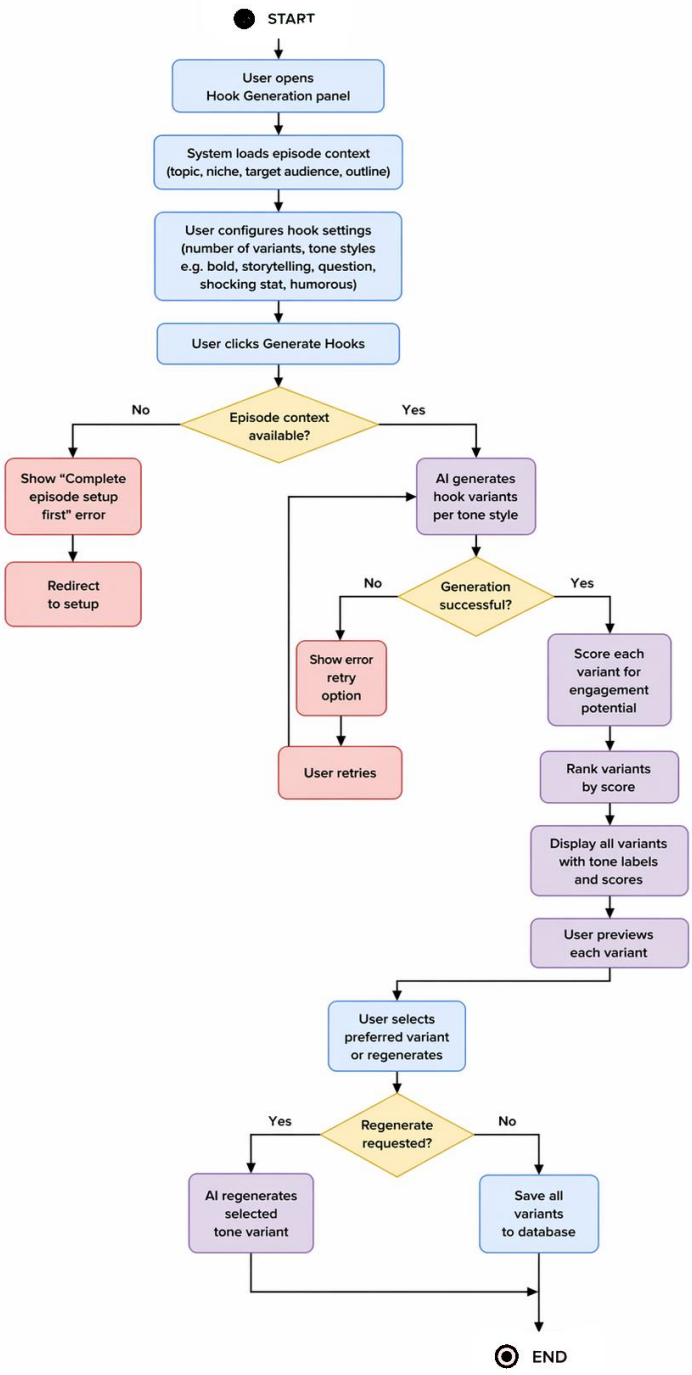
Key Elements

- User opens Timestamp Generator → system loads episode data (questions, segments, total duration).
- User sets target episode duration → configures segment weight percentages (intro %, main discussion %, guest Q&A %, outro %).
- Clicks Generate Timestamps → check if episode duration set. If not → prompt user.
- Verify segments available. If missing → "No segments found" error.
- Valid inputs → calculate time per segment based on configured weights → distribute total duration proportionally.
- Any segment below minimum threshold (30 seconds) → auto-adjust using cascade update → recalculate timestamps.
- Sum of segment times ≠ total duration → adjust final segment to fill gap.
- Display formatted timestamp entries (e.g., 00:00 Intro, 05:30 Guest Background).
- User reviews and edits as needed.
- If approved → save to database → cascade to update episode outline and question assignments.
- If rejected → user manually adjusts before re-approving.

Contribution

- Timestamping is a professional production standard rarely automated in planning tools.
- Cascade update logic and minimum-threshold enforcement ensure practical production quality.
- Generated structure is usable out-of-the-box without manual correction.

Module 5: UC1 Generate Multiple Hook Variants:



Overview

- Models the Hook Suggester Agent's workflow.
- Generates multiple stylistically varied episode opening hooks, scores them, and presents for user selection or regeneration.

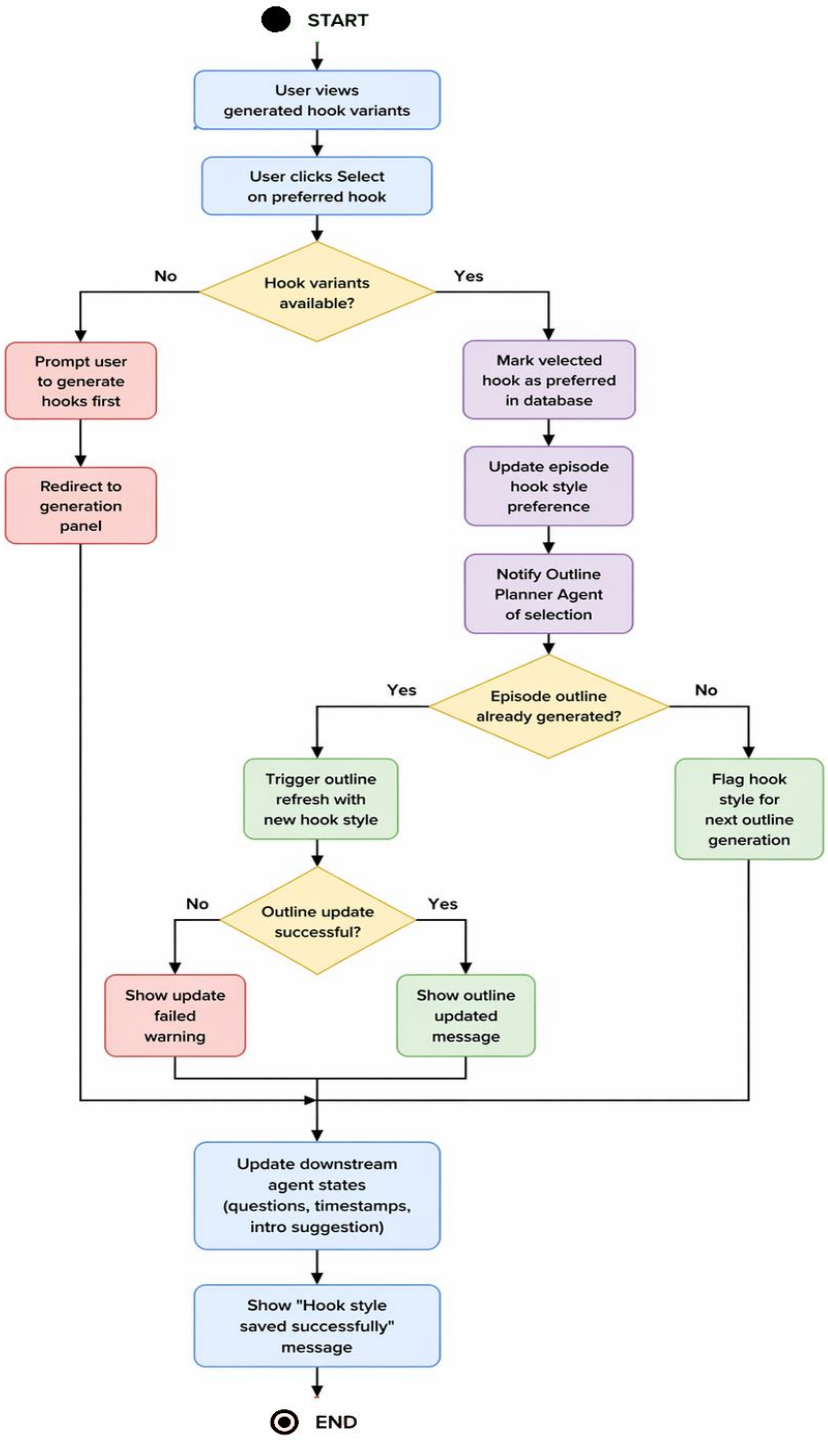
Key Elements

- User opens Hook Generation panel → system loads episode context (topic, niche, target audience, outline).
- User configures: number of variants, tone styles (bold, storytelling, question-based, shocking statistic, humorous) → clicks Generate Hooks.
- Episode context missing → "Complete episode setup first" error → redirect to setup.
- Context present → AI generates hook variants per requested tone styles.
- Generation success check. Failure → error + retry option.
- Success → each variant scored for engagement potential → ranked by score.
- Display ranked variants with tone labels and engagement scores → user previews each.
- User either selects preferred variant or requests regeneration of specific tone style.
- Regenerate → AI re-generates only selected tone variant.
- Keep selection → save all variants to database.

Contribution

- System produces ranked set of stylistically diverse alternatives, not just one hook.
- Scoring and ranking mechanism guides user toward high-quality choices.
- Retains creative control while improving episode quality without replacing human judgement.

Module 5: UC2 Select Preferred Hook Style:



Overview

- Models the hook selection workflow after hook variants are generated.

- Shows how user commitment to a preferred style propagates as state updates to the Outline Planner Agent and downstream components.

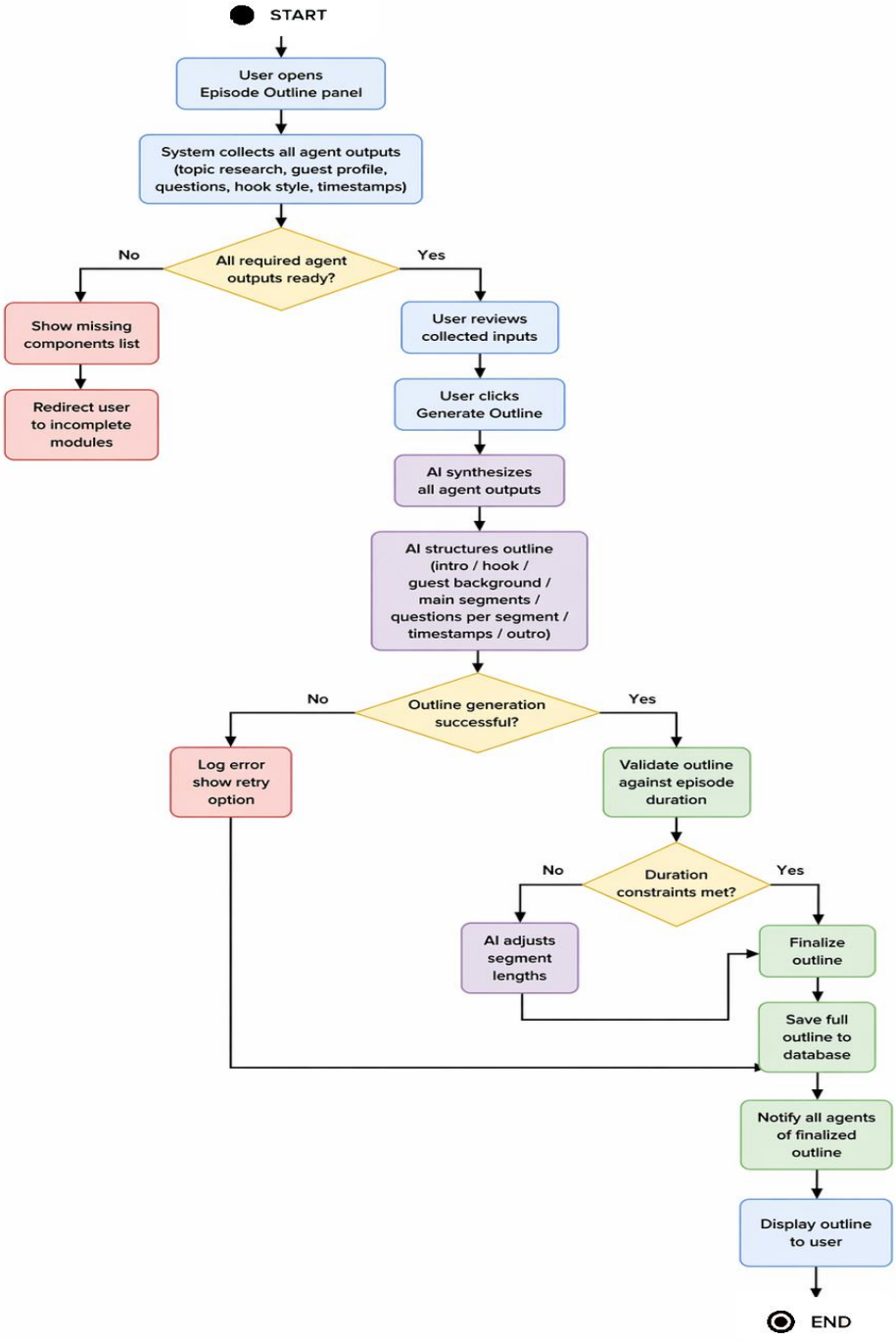
Key Elements

- User clicks Select on preferred hook from hook variants display.
- System verifies hook variants exist. If not → prompt user to generate hooks first → redirect to generation panel.
- Variants available → mark selected hook as preferred in database → update episode's hook style preference.
- Notify Outline Planner Agent of selection.
- Check if episode outline already generated.
- If outline exists → trigger outline refresh incorporating new hook style.
- Success → "Outline updated" message. Failure → "Update failed" warning.
- If no outline yet → flag selected hook style for incorporation in next outline generation.
- Update downstream agent states (questions, timestamps, introduction suggestions) to align with chosen style.
- Display "Hook style saved successfully" confirmation.

Contribution

- Exposes inter-agent communication architecture.
- Single user preference propagates across entire planning pipeline.
- Explicit agent-notification and cascade update maintain consistency across generated artefacts.

Module 6: UC1 Generate Full Episode Outline:



Overview

- Models the Outline Planner Agent's synthesis workflow — the culminating step of PodCraft's agentic pipeline.

- Describes how all upstream agent outputs are aggregated, structured, and validated into a single coherent episode outline.

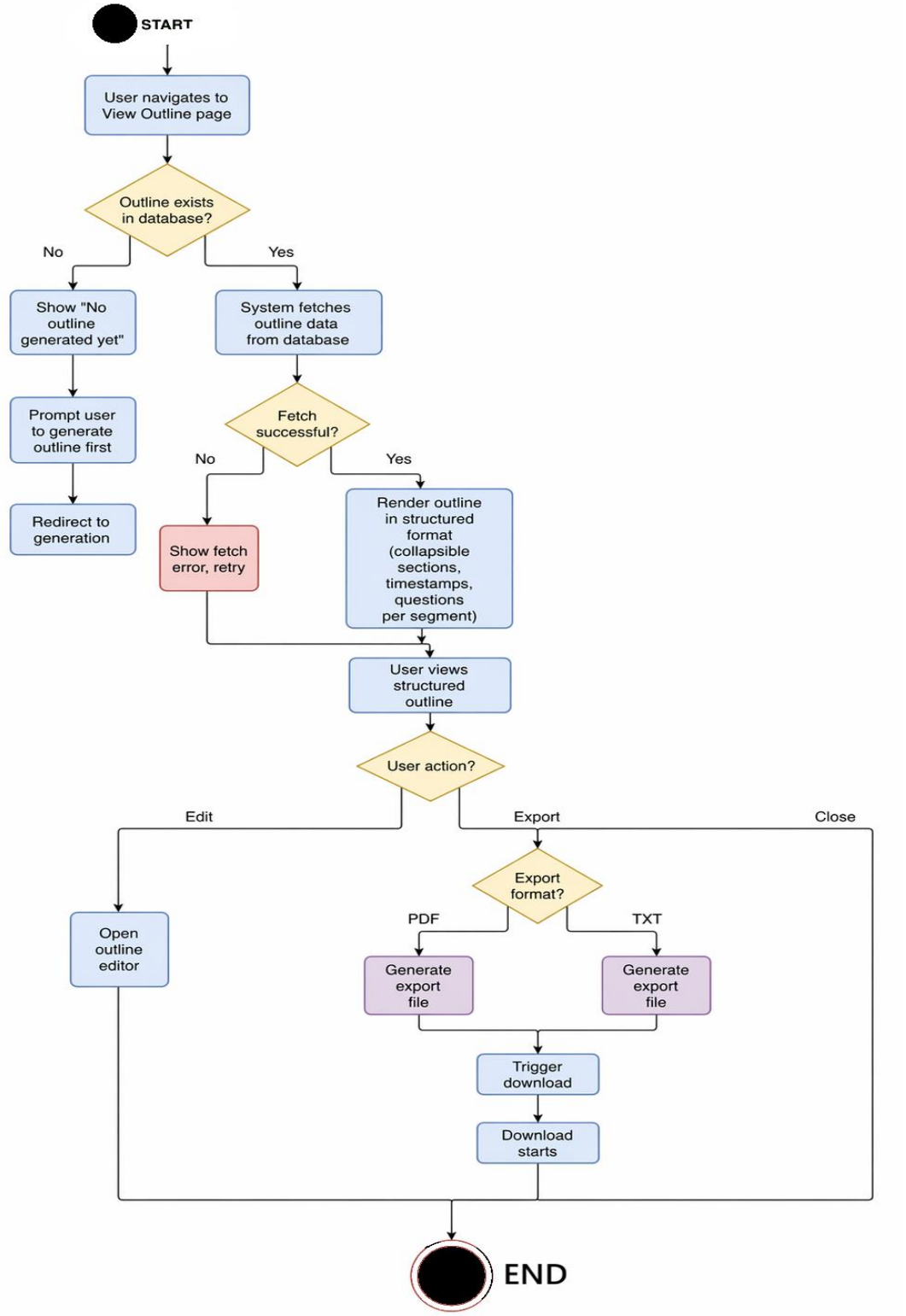
Key Elements

- User opens Episode Outline panel → system collects all agent outputs: topic research, guest profile analysis, generated questions, selected hook style, timestamp structure.
- Check if all required agent outputs ready. Missing any → show list of missing components → redirect user to complete relevant modules.
- All inputs present → user reviews collected data → clicks Generate Outline.
- AI synthesises all agent outputs → structures outline across fixed sections: introduction, hook, guest background, main discussion segments (with questions per segment + timestamps), outro.
- Generation success check. Failure → log error → offer retry.
- Success → validate outline against episode duration constraints.
- Constraints not met → AI automatically adjusts segment lengths.
- Constraints satisfied → finalise outline → save to database → notify all agents of finalised state → display complete outline to user.

Contribution

- Most architecturally significant diagram in the system.
- Demonstrates transition from discrete agent outputs to unified coherent planning artefact.
- Dependency check enforces pipeline sequencing constraints, ensuring final outline is grounded in complete and consistent upstream data.

Module 6: UC2 View Outline in Structured Format:



Overview

- Models the read, render, and export workflow for the episode outline.

- Describes retrieval from database, structured interactive rendering, and user choices to edit, export, or close.

Key Elements

- User navigates to View Outline page.
- System checks if generated outline exists in database.
- No outline found → show "No outline generated yet" message → prompt user to navigate to generation panel.
- Outline record exists → fetch outline data.
- Fetch fails → error with retry option.
- Fetch succeeds → render outline in structured interactive format with collapsible sections, timestamps, questions per segment.
- User views outline → reaches three-way action decision: Edit, Export, or Close.
- Edit → open outline editor.
- Export → trigger format selection (PDF or TXT) → generate corresponding export file → trigger browser download → download begins.
- Close → end session without further action.

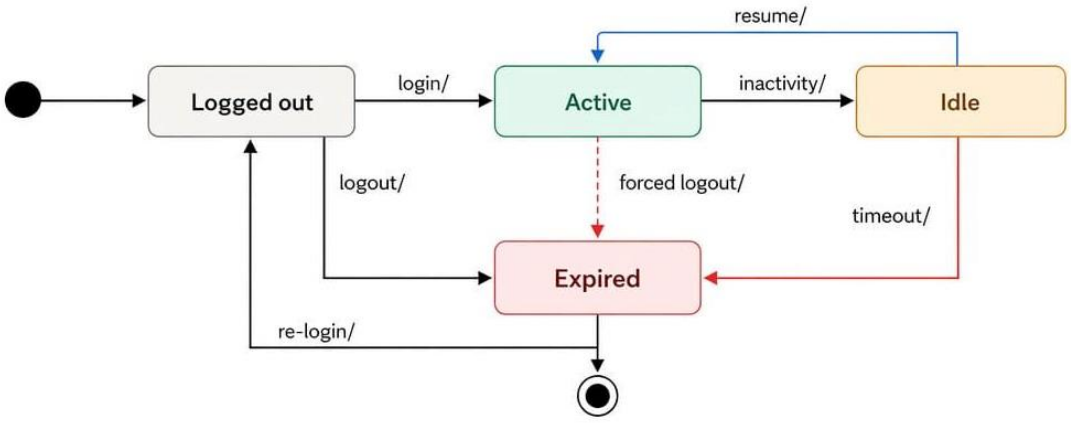
Contribution

- Defines how PodCraft delivers final output to the user.
- Multi-format export capability (PDF and TXT) meets real-world production needs (sharing with co-hosts, uploading to production tools).
- Collapsible structured rendering reflects progressive disclosure UI principle, allowing users to focus on one section at a time.

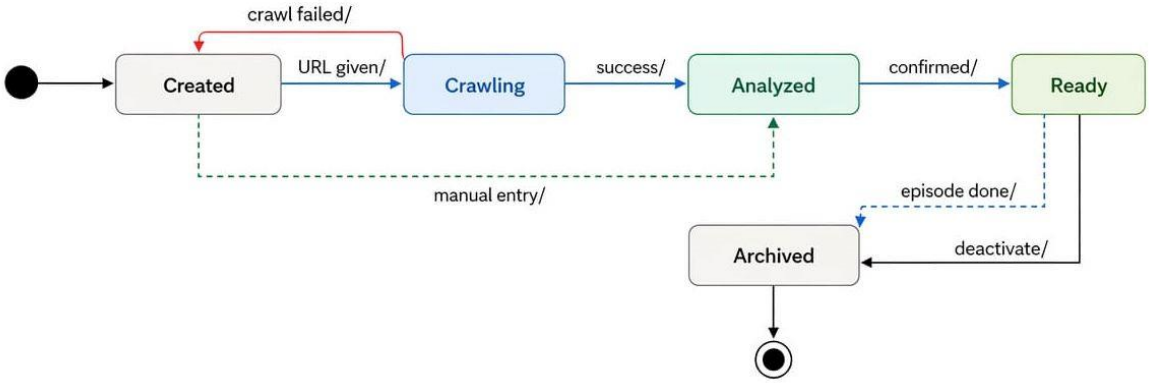
3.3.4 State Diagrams:

Batch 1: User Session and Guest Profile.

User session

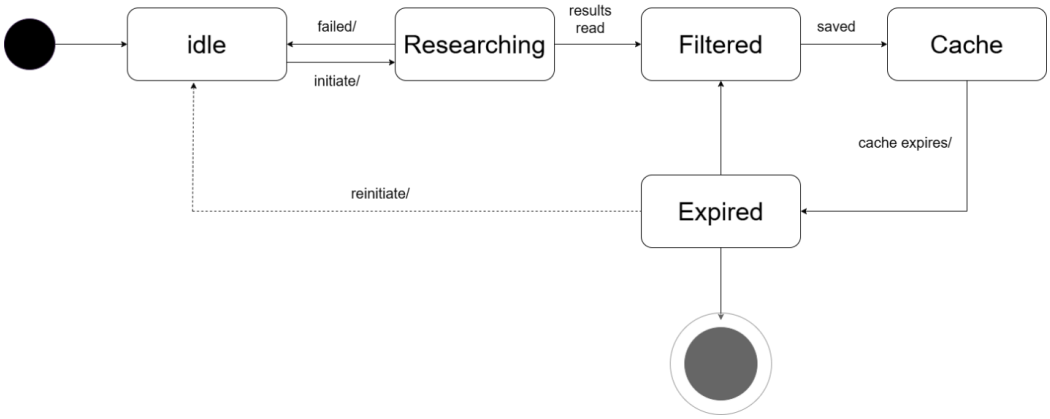


Guest profile

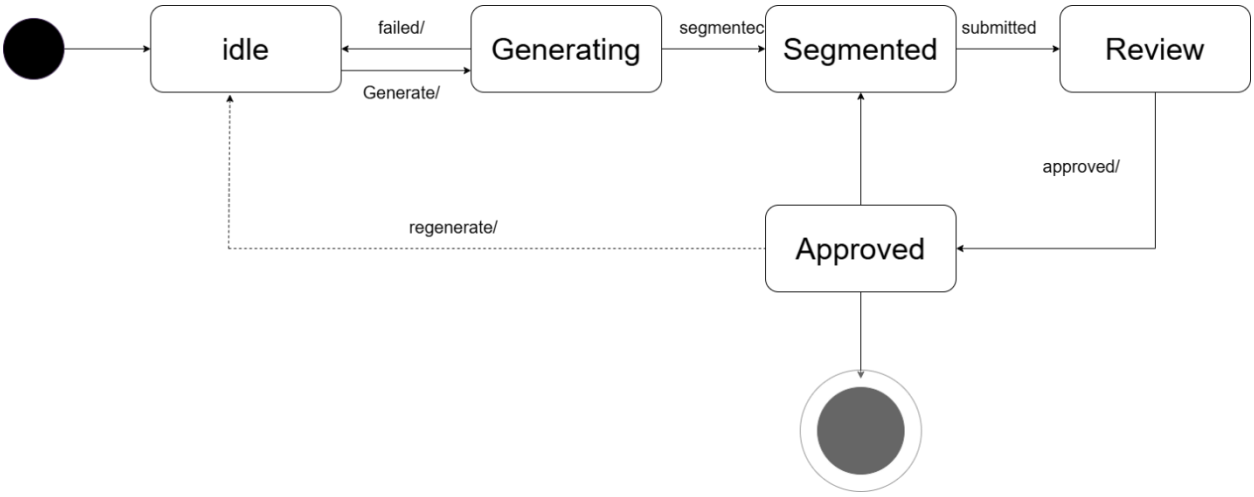


Batch 2: Topic Research Agent and Question Generator Agent:

Topic research agent

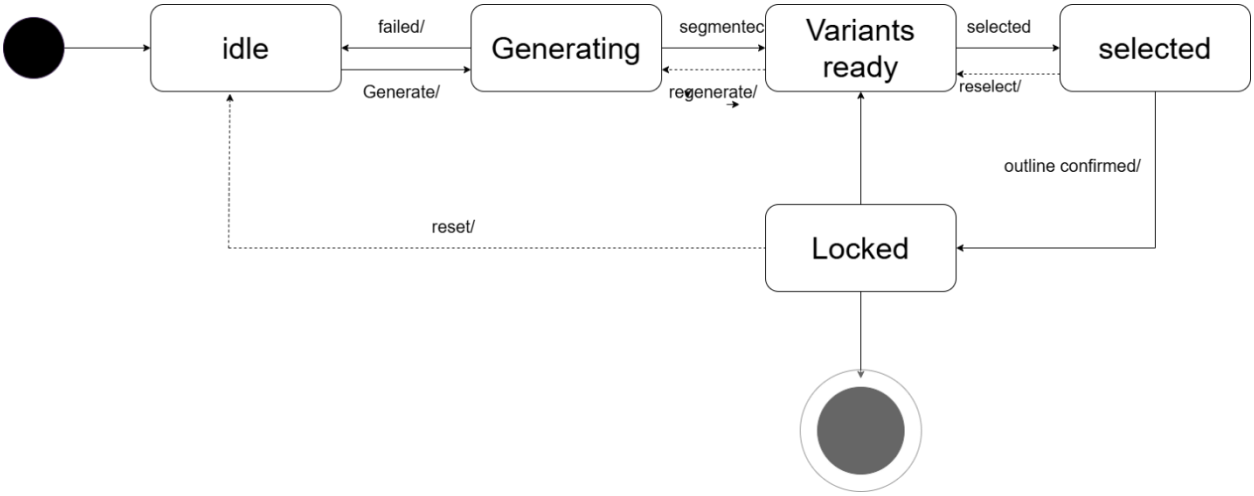


Question generator agent

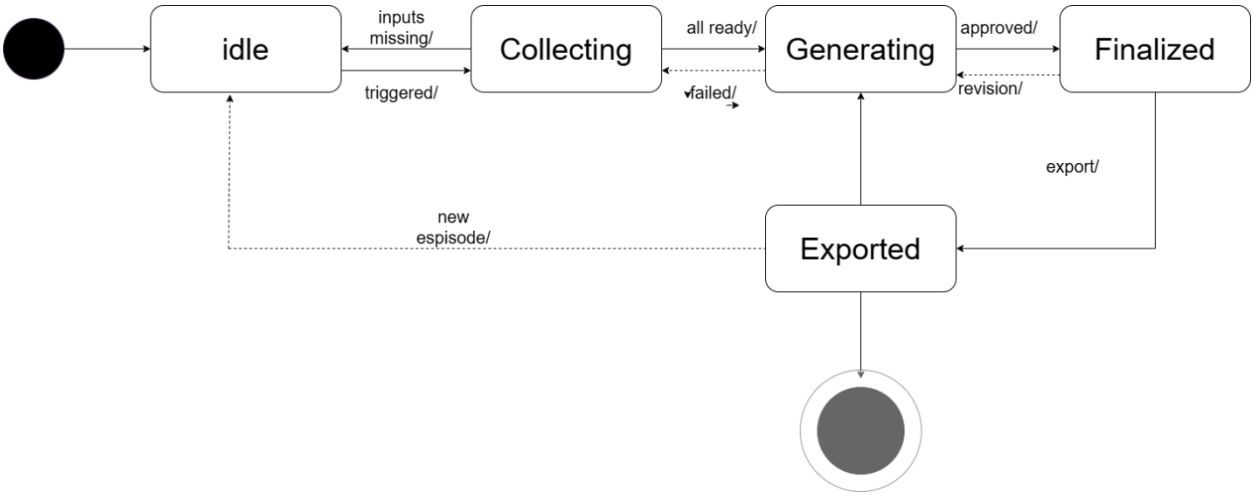


Batch 3: Hook Suggestion Agent and Episode Outline Agent:

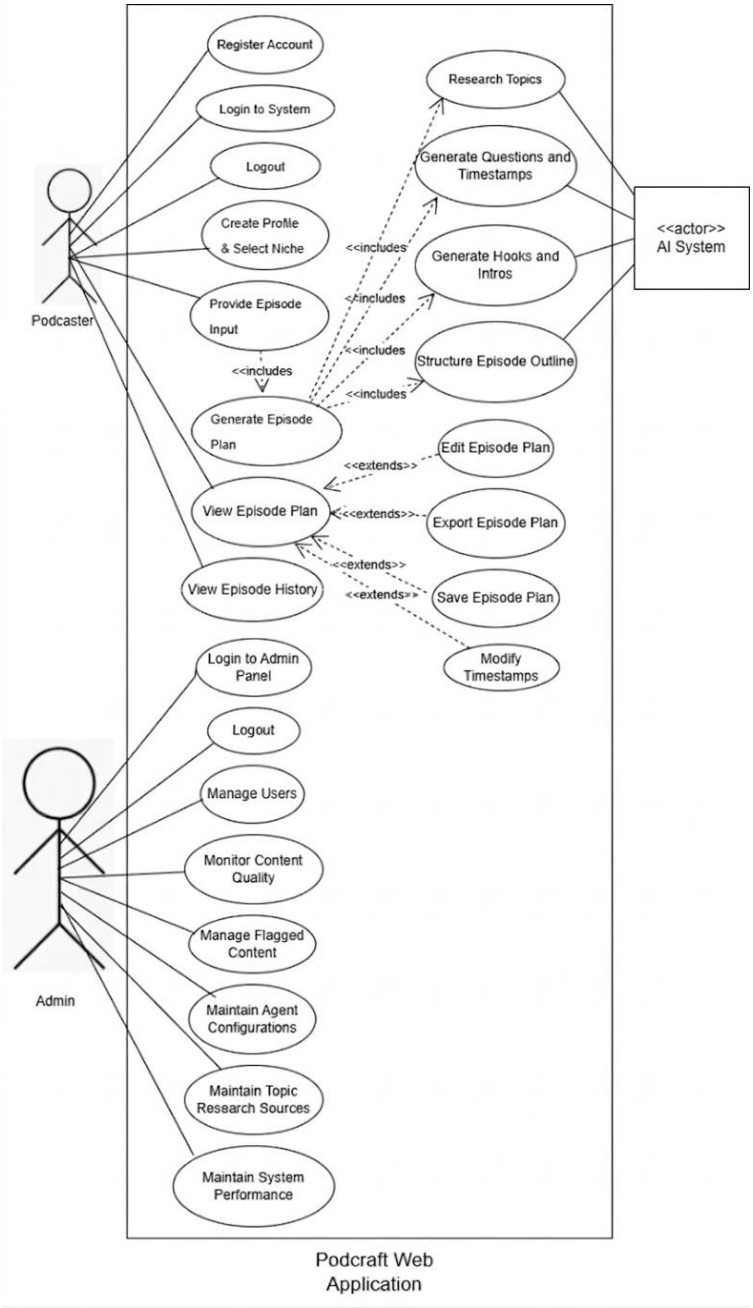
Hook suggestion agent



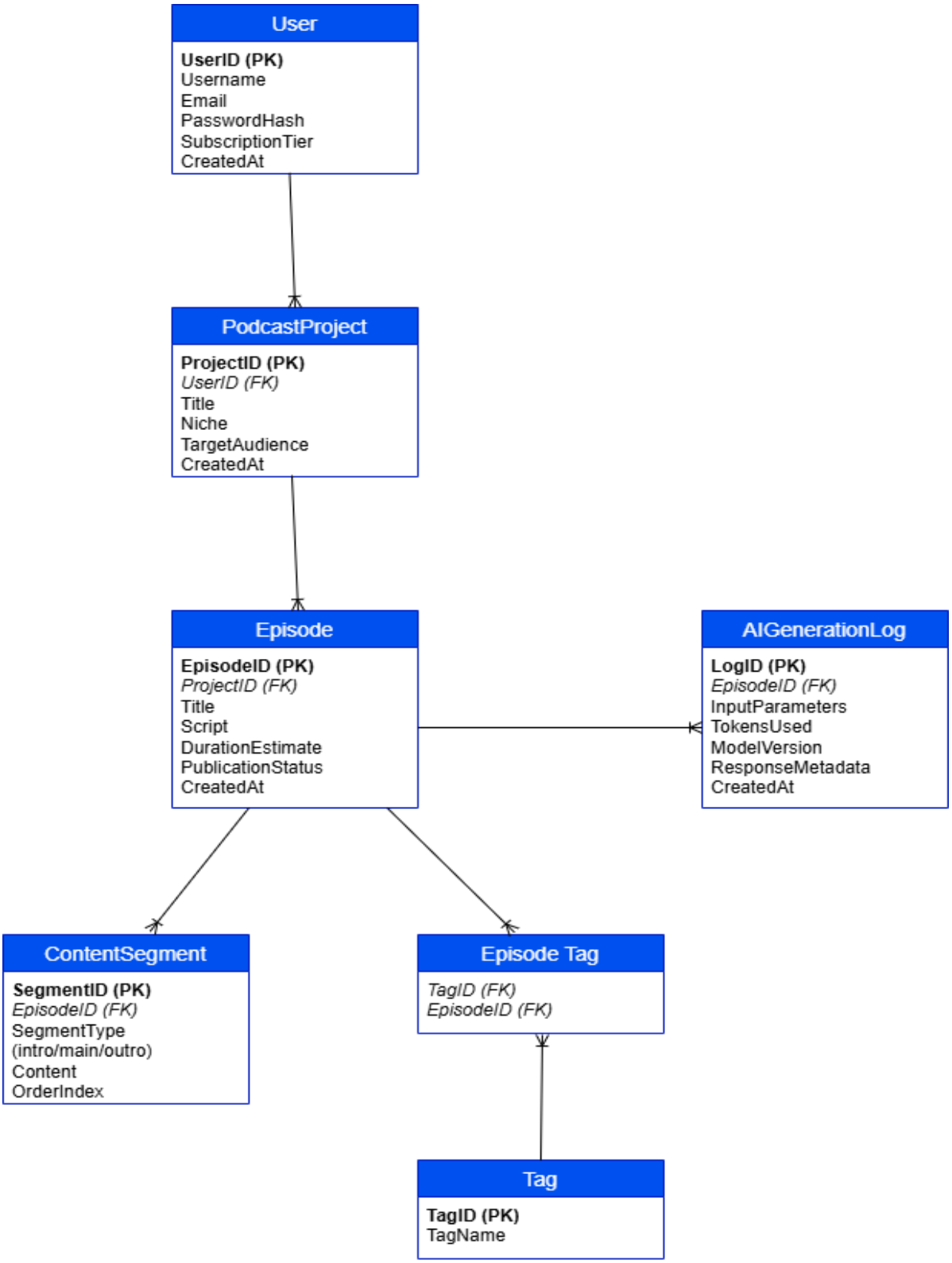
Episode outline agent



3.3.5 UseCase Diagram:



3.4 Data Design



The data design of PodCraft is modeled around six core entities that together capture the full lifecycle of podcast creation and user interaction within the system. The User entity serves as the central node, storing authentication credentials, profile information, and subscription tier. Each user can own multiple PodcastProject records, each of which encapsulates a distinct podcast concept, including its title, niche, target audience, and creation timestamp.

A PodcastProject is composed of one or more Episode records, which store AI-generated scripts, episode titles, duration estimates, and publication status. The Episode entity maintains a many-to-one relationship with its parent project and a one-to-many relationship with ContentSegment records, which represent individual sections such as introductions, main discussions, and outros. The Tag entity provides a many-to-many associative relationship with Episodes, enabling flexible content categorization and search. Finally, the AIGenerationLog entity records every invocation of the AI engine, capturing input parameters, token usage, model version, and response metadata. This log is critical for monitoring system performance, auditing generated content, and supporting future fine-tuning efforts. Together, these entities provide a normalized, scalable schema that supports the full range of PodCraft's agentic content generation features.

Work Division:

Asna Hammad (FA23-BDS-051)

- Section 3.2, Architecture Pattern (whole)
- Section 3.3.1 Sequence diagrams (all)
- Section 3.3.2 class diagram
- Section 3.3.5 usecase diagram
- document formatting

Awais Ashraf (FA23-BDS-053)

- Section 3.1 System architecture overview (whole)
- Section 3.3.3 Activity diagrams (all)
- Section 3.3.4 State Diagrams (all)
- Section 3.4 (whole)